

SAINT MARY'S
UNIVERSITY SINCE 1802

One University. One World. Yours.

PROJECT REPORT

MCDA 5585 and MCDA 5586

Intelligent Automation and Reliability Engineering

Submitted By:

Bhavik Kantilal Bhagat

A00494758

under the guidance of:

Dr. Pawan Lingras

Mr. Kishor Deotale

in partial fulfillment for the awarding of the degree:

Master of Science in
Computing and Data Analytics

August 2026

ACKNOWLEDGEMENT

I would like to express my heartfelt appreciation to everyone who contributed to the successful completion of this major project report. Their guidance, support, and encouragement were pivotal to its success.

I am especially grateful to **Dr. Pawan Lingras**, Academic Supervisor and Director of the Master of Science in Computing and Data Analytics (MCDA) program at Saint Mary's University, for providing me with this invaluable opportunity. His constant support and guidance were crucial throughout the duration of this project.

I extend my deepest gratitude to my manager and industry supervisor, **Mr. Kishor Deotale**, at Citco Technology Management (CTM). His mentorship gave me the opportunity to explore new technology stacks that had not previously been adopted for SRE at Citco — such as AWS-native tooling (CloudWatch, X-Ray, Application Signals) and Grafana as alternatives to Dynatrace — and to help lay the groundwork for a more modern observability approach. This involved assessing the benefits, limitations, and cost implications of each tool to shape a sustainable path forward: one that offers an “eagle-eye” view of the infrastructure, enabling both reactive detection of issues as they arise and proactive identification through warning and critical alerts.

Special thanks to senior CTM team members **Ho Hoi Leung (Horace)** and **Kristia Marie Labos (Kri)** for their hands-on mentorship, technical insights, code review and continuous feedback for improvements. I also express my sincere appreciation to my SMU MCDA colleagues and CTM team members, **Sridhar Vadla** and **Soundarya Venkataraman**, for their active collaboration and shared commitment to technical excellence.

Finally, I would like to thank both **Saint Mary's University** and **Citco Technology Management (CTM)** for giving me this incredible opportunity to gain industry exposure and work on such a sophisticated real-world financial technology project.

CERTIFICATE

This is to certify that **Bhavik Kantilal Bhagat** (Student ID: **A00494758**), a student at Saint Mary's University in the Master of Science in Computing and Data Analytics program, has completed the projects in fulfillment of the course requirements for **MCDA 5585** and **MCDA 5586** on the project titled "**Intelligent Automation and Reliability Engineering for Financial Services**".

This project represents a total of **1020 hours** (900 internship hours + 120 academic hours) of technical and project work conducted from Mar 15 to Aug 31, 2026.

.....
Dr. Pawan Lingras

ACADEMIC SUPERVISOR
DEPARTMENT OF MATHEMATICS
AND COMPUTING SCIENCE
SAINT MARY'S UNIVERSITY
923 Robie St, Halifax,
NS, Canada B3H 3C3

.....
Mr. Kishor Deotale

INDUSTRY SUPERVISOR / MANAGER
CITCO TECHNOLOGY MANAGEMENT (CTM)
CITCO (CANADA) INC.
5151 George St, Halifax,
NS, Canada B3J 1M5

Contents

Acknowledgement	i
Certificate	ii
1 About Company & Organization	1
1.1 The Citco Group of Companies	1
1.1.1 Core Service Divisions	1
1.1.2 Industry Recognition	2
1.2 Citco Technology Management — Halifax, Canada	2
1.2.1 Team Structure & Responsibilities	2
1.2.2 My Role	4
2 Executive Summary	5
3 Tools & Technologies	7
3.1 AWS Cloud Services	7
3.2 Observability & Monitoring	9
3.3 Development Tools	9
3.4 AI-Native Development Tools	11
3.5 RPA Platforms	11
4 Project Overview	11
4.1 Project Background and the Motivating Incident	11
4.1.1 Pre-Internship Monitoring Posture	11
4.1.2 CAIS-Pricing Silent Outage (Jan–Feb 2026)	12
4.1.3 The Meridian Incident (Apr 2026)	12
4.1.4 From Incident to Programme	12
4.2 Project Overview	12
4.2.1 Work Stream 1: CloudWatch Dashboards and IA Infrastructure Monitoring	13
4.2.2 Work Stream 2: Grafana AMG Centralized Dashboard	13
4.2.3 Work Stream 3: IA Infrastructure Inventory API	14
4.2.4 Work Stream 4: RPA Production Support	15
5 Project Goals & Deliverables	16

5.1	Project Goals	16
5.2	Deliverables and Technical Scope	17
5.2.1	IA Infrastructure Monitoring Platform (IISS-435, 455, 461, 469, 482-486)	17
5.2.2	Centralized Grafana SRE Dashboard (IISS-487)	18
5.2.3	IA Infrastructure Inventory API (IISS-507-510)	19
5.3	Acceptance Criteria	19
5.3.1	IA Infrastructure Monitoring Platform	19
5.3.2	SRE Observability Platform	19
5.3.3	IA Infrastructure Inventory API	20
5.3.4	RPA Production Support	20
6	Learning Goals	21
6.1	Technical Learning Goals	21
6.2	Non-Technical Learning Goals	22
7	Methodologies & Requirements	23
7.1	SRE Principles	23
7.1.1	The Four Golden Signals	24
7.1.2	Service Level Indicators and Objectives	24
7.1.3	Error Budgets	25
7.2	Observability Gap Analysis	25
7.2.1	Pre-Internship Monitoring Posture	26
7.2.2	Gap Analysis Summary	27
7.3	Dynatrace Parity Analysis	27
7.4	Limitations and Mitigations	28
8	Requirements Elicitation	29
8.1	Stakeholder Identification	29
8.2	Requirements Elicitation	29
8.2.1	Elicitation Methods	30
8.2.2	Key Insights	30
8.3	Functional and Non-Functional Requirements	31
8.3.1	Functional Requirements	31
8.3.2	Non-Functional Requirements	31
8.3.3	Requirements Traceability	31

8.3.4	Out-of-Scope Requirements	33
9	Implementation & Architecture	34
9.1	Platform Overview: Four-Layer Architecture	34
9.2	CloudWatch Dashboard Platform (IISS-455/469/482-486)	35
9.2.1	Initial Lambda Dashboard (IISS-455)	35
9.2.2	Extended Dashboard and Architecture Analysis (IISS-469)	36
9.2.3	IA Infrastructure Monitoring Sub-Stories (IISS-482 to IISS-486)	36
9.3	Grafana AMG Observability Platform (IISS-487)	38
9.3.1	Architectural Layers	38
9.3.2	Grafana Version Upgrade: Architectural Resilience Test	42
9.4	IA Resources Inventory API (IISS-507)	42
9.4.1	Technology Stack	43
9.4.2	Discoverer Registry Pattern	43
9.4.3	API Endpoints	44
9.4.4	Tag-Based Discovery and the Two-Stage Filter	44
9.4.5	Async Architecture	45
9.4.6	Test Strategy	46
9.5	Design Decisions & Tradeoffs	46
9.5.1	Decision 1: Secrets Manager 64 KB Limit → S3 Migration	47
9.5.2	Decision 2: AppName Tag as Mandatory Secondary Discriminator	48
9.5.3	Decision 3: CloudWatch as Sole Data Source (No Prometheus)	48
9.5.4	Decision 4: Lambda-Backed IaC Provisioner vs. Static CloudFormation	49
9.6	Phase 1 — Onboarding and CloudWatch (Weeks 1-7, Mar 15 – May 10)	50
9.6.1	Part-Time Onboarding (Mar 15 – Apr 12)	50
9.6.2	Training Week (Apr 13-18)	51
9.6.3	First Deliverables — IISS-455, IISS-461, IISS-469 (Apr 19 – May 10)	52
9.7	Phase 2 — Grafana Intensive (Weeks 8-11, May 6 – May 27)	53
9.7.1	Elevation to Critical and Week 8 Support Shift	53
9.7.2	Infrastructure Deployment Day — May 11	53
9.7.3	Grafana 10.4 to 12.4 Upgrade — May 13	54
9.7.4	Dashboard Build — May 11-27 (80+ Hours)	54
9.8	Phase 3 — Dashboard Completion and SQS Feature (Weeks 12-15, May 28 – Jun 18)	56
9.8.1	Rows 7-14 and the Deep-Link Pattern (May 28 – Jun 4)	56

9.8.2	IISS-482: SQS Feature — Jun 12	57
9.8.3	Final Deployment — Jun 18, Version v728	57
9.9	Phase 4 — Inventory API and Refinements (Weeks 16–22, Jun 19 – Aug 31)	57
9.9.1	IISS-507: Scaffolding the Inventory API — Jun 11	58
9.9.2	Inventory API Development — Jun 11 – Jul 14	58
9.9.3	Test Suite Milestone — Jul 14: 562 Tests at 99% Coverage	59
9.9.4	X-Ray Tracing Research — Jul–Aug	59
9.10	RPA Support Operations	60
9.10.1	Support Rotation Model	60
9.10.2	Notable Incidents	61
9.10.3	Support Incident Summary	63
9.10.4	Cross-Cutting Learning: How Support Incidents Shaped Dashboard Design	63
9.11	Development Repositories	64
9.11.1	Primary Repositories	65
9.11.2	Full Project Scope and Tracking	65
10	Weekly Breakdown	66
10.1	Phase 1: Foundation & CloudWatch Dashboards (Weeks 1–7, Mar 15 – May 10)	66
10.1.1	Weeks 1–4: Onboarding and First Deliverables (Mar 15 – Apr 11)	66
10.1.2	Phase 1 Total: 100 hours — Key Insight	68
10.2	Phase 2: Grafana AMG Intensive Build (Weeks 8–11, May 6 – May 27)	68
10.2.1	Phase 2 Total: 157 hours — Key Milestone	68
10.3	Phase 3: Dashboard Completion, Hotfixes & SQS (Weeks 12–15, May 28 – Jun 18)	68
10.3.1	Phase 3 Total: 160 hours — Key Milestone	69
10.4	Phase 4: Inventory API, X-Ray Research & Report (Weeks 16–24, Jun 19 – Aug 31)	69
10.4.1	Phase 4 Total: 480 hours — Key Milestone	70
10.5	Internship Hours Summary	70
11	Conclusions	70
11.1	Learning Goals Evaluation	70
11.2	Conclusion and Future Work	70
11.2.1	Summary of Outcomes	70
11.2.2	Personal Reflection	71
12	Achievements & Business Value	72

12.1	Key Deliverables and Metrics	72
12.1.1	IISS-487: Grafana AMG Centralized SRE Dashboard	73
12.1.2	IISS-482/455/469: CloudWatch Monitoring Platform	73
12.1.3	IISS-507: IA Resources Inventory API	74
12.1.4	RPA Production Support	74
12.2	Impact and Business Value	74
12.2.1	Incident Detection Improvement: MSK Consumer Lag	74
12.2.2	Operational Efficiency: Fleet Visibility	75
12.2.3	RPA Support Value	75
13	Challenges & Mitigations	76
13.1	Challenges and Mitigations	76
13.1.1	Challenge 1: CloudWatch SEARCH() Visibility Gap	76
13.1.2	Challenge 2: Metric Insights SQL Cannot Compute Percentiles	76
13.1.3	Challenge 3: Grafana 10.4 → 12.4 Mid-Development Upgrade	77
13.1.4	Challenge 4: Secrets Manager 64 KB Limit	77
13.1.5	Challenge 5: Shared BudgetCode Tag Ambiguity	77
13.1.6	Challenge 6: IAM Permission Boundary Blocking Subscription Filters	78
13.1.7	Challenge 7: Module-Import Credential Caching (IISS-706 Hotfix)	78
14	Future Work	78
A	Appendix: Hourly Activity Logs Summary	82

List of Figures

9.1	IA-SRE Observability Platform — High-Level Architecture (four layers)	35
-----	---	----

List of Tables

5.1	Project Deliverables — JIRA Scope	17
7.1	Observability Gap Analysis — Pre- vs. Post-Internship	27
7.2	Dynatrace Capability Parity — AWS-Native Stack vs Dynatrace (selected features)	28
8.1	Stakeholder Identification	29
8.2	Functional Requirements	32
8.3	Non-Functional Requirements	33
9.1	Grafana AMG Dashboard — Row Structure (Rows 1–15 shown; Rows 16–17: Alarms & Notifications, X-Ray Traces)	84
9.2	IA Resources Inventory API — Endpoint Summary	85
9.3	Design Decision Summary	85
9.4	Support Incident Summary	86
9.5	Full JIRA Scope — Internship Deliverables	87
10.1	Internship Hours by Phase	88
11.1	Learning Goals — Self-Evaluation at Internship Completion	89
12.1	IISS-487 Grafana Dashboard — Delivery Metrics	90
12.2	IISS-507 Inventory API — Delivery Metrics	90
14.1	Auto-Healing Agent — Initial Remediation Catalogue	91
A.1	Summary of internship hours by phase.	91

Chapter 1. About Company & Organization

1.1 The Citco Group of Companies

The Citco Group of Companies (Citco) is the world's largest independent hedge fund administrator, with over \$1.8 trillion in assets under administration. Originally founded in 1948 as the *Curaçao International Trust Company* in Curaçao, Netherlands Antilles, the firm is currently headquartered in Tortola, British Virgin Islands. Over more than seven decades, Citco has grown from a civil law notary practice serving private clients and multinationals into a global financial services conglomerate operating across 50+ offices worldwide with approximately 16,000 employees spanning North America, Europe, Asia-Pacific, and the Caribbean.

Citco remains majority independently owned, a distinction that underpins its client-centric operating philosophy. The firm's CEO, Christopher Smeets, leads a management team that has overseen sustained organic growth — from \$250 billion in administered assets in 2005 to the current \$1.8 trillion benchmark. This growth trajectory reflects Citco's pioneering role in shaping the fund administration sector and its continuous investment in proprietary technology platforms to service increasingly complex alternative investment structures.

1.1.1 Core Service Divisions

Citco's global operations are organized across several specialized divisions:

- **Citco Fund Services (CFS) & Investor Relations** — The flagship division and global leader in services for the alternative investment industry. CFS provides full-service fund administration, fund accounting, net asset value (NAV) calculations, investor relations, and reporting for over 2,000 funds. CFS has maintained SOC 1 Type 2 certification for 22 consecutive years and achieved its inaugural SOC 2 Type 2 certification in 2025, underscoring its commitment to operational controls and data security.
- **Citco Governance Services** — Provides corporate and global subsidiary governance, secretarial services, financial accounting and reporting, tax and audit liaison, entity incorporation, and market entry services across multiple jurisdictions.
- **Citco Banking** — Offers clients a comprehensive suite of banking services including investor accounts, operational and deposit bank accounts, foreign exchange, and credit services.

- **Citco Technology Management (CTM)** — The internal technology arm responsible for developing and maintaining Citco's proprietary technology ecosystem, cloud infrastructure, data platforms, and enterprise applications that power fund administration and middle-office operations globally.
- **Middle Office Solutions** — Encompassing Treasury and Collateral Management, OTC settlements, Trade Matching, and Portfolio Finance. Citco's cloud-based *Æxeo*® Treasury platform provides centralized wire approval and fund movement visibility.

1.1.2 Industry Recognition

Citco has consistently received industry accolades, including the Hedgeweek European Award for Fund Administrator of the Year (Middle Office Services, 2026), Best Administrator over \$30 billion (APAC Hedge Fund Services Awards, 2026), and Fund Administrator of the Year for GPs over \$30 billion (Private Equity Wire US Awards, 2025). These awards reflect both depth of service and breadth of technological innovation.

1.2 Citco Technology Management — Halifax, Canada

The internship placement is within **Citco Technology Management (CTM)**, the technology division of the Citco Group, based at the Halifax, Nova Scotia office located at 5151 George Street. The Halifax office is a significant operational centre for Citco Canada Inc., occupying multiple floors of the building and serving as a hub for technology development, fund administration services, and operational support for North American and global clients.

CTM is responsible for the design, development, deployment, and reliability of Citco's internal technology platforms. This includes enterprise application development, cloud infrastructure management, DevOps and Site Reliability Engineering (SRE), data engineering pipelines, and intelligent automation initiatives. The Halifax CTM team collaborates with Citco colleagues across global offices to deliver technology solutions that underpin the firm's fund administration, investor relations, and governance services.

1.2.1 Team Structure & Responsibilities

My placement is within the **Intelligent Automation & Site Reliability Engineering (IA-SRE)** team under the supervision of Mr. Kishor Deotale (Manager, CTM). The team is co-located across Halifax

(Canada) and Hyderabad (India), with stakeholders spanning Dublin and London. The internship-period team comprised the following members:

- **Mr. Kishor Deotale** — Manager and Industry Supervisor. Kishor holds overall accountability for the IA-SRE mandate, guides technical direction, and serves as the liaison between Halifax engineering and Citco's global stakeholder groups. He co-leads the RPA support programme and sets engineering priorities across all work streams.
- **Ho Hoi Leung ("Horace")** — Senior Mentor, RPA Support. Horace is the primary knowledge holder for Blue Prism and UiPath platform operations, responsible for escalation triage in the support model and for day-to-day BOT reliability. He delivered the RPA support knowledge-transfer sessions and acts as the first escalation point for production incidents.
- **Kristia Marie Labos ("Kri")** — Senior Mentor, SRE. Kri leads the SRE practice for the team, including the four golden signals framework. She guided the observability platform design and provided feedback on alarm threshold selection throughout the dashboard build.
- **Sridhar Vadla and Soundarya Venkataraman** — Co-interns (SMU MCDA, Mitacs BSI IT49539). Both joined the Halifax team concurrently, each working on complementary work-streams of the IA monitoring and automation platform, enabling parallel progress across the observability roadmap.

The IA-SRE team's mandate encompasses:

- Defining and maintaining Service Level Indicators (SLIs) and Service Level Objectives (SLOs) for critical financial infrastructure, translating business reliability expectations into measurable engineering targets.
- Designing and operating the centralized observability platform — Amazon Managed Grafana (AMG) with CloudWatch as the primary data source — that provides unified visibility across all AWS services in the IA-IT-SRE Infrastructure.
- Authoring parameterized CloudFormation and AWS SAM templates for alarm provisioning, developing Lambda-based dynamic provisioners, and maintaining the infrastructure code that deploys and manages AWS services consistently across the six platforms.
- Developing intelligent automation solutions including AI-driven incident prediction, log anomaly detection, and self-healing infrastructure capabilities.

- Ensuring the reliability and recovery of Robotic Process Automation workloads running on Blue Prism and UiPath Cloud/on-premises platforms, including production support rotations and incident response.

1.2.2 My Role

As a participant in the Mitacs Business Strategy Internship (BSI) program, I was embedded within the **Intelligent Automation & Site Reliability Engineering (IA-SRE)** team at CTM Halifax. I collaborated closely with my manager Kishor Deotale, senior mentors Horace and Kri, co-interns Sridhar Vadla and Soundarya Venkataraman, and stakeholders across Halifax, Hyderabad, and Dublin. Weekly standups, sprint reviews, and direct Confluence and JIRA-based collaboration formed the day-to-day working rhythm.

My responsibilities spanned the following areas:

- **IA Infrastructure Monitoring:** Developing per-platform CloudWatch dashboards for Lambda, ECS, EC2, SQS, EKS, and related AWS services across all six platforms, establishing the architectural patterns for dynamic resource discovery (IISS-455, IISS-469, IISS-482–486).
- **SRE Observability Platform Development:** Designing and building the centralized Amazon Managed Grafana (AMG) dashboard covering all AWS services across the six IA-IT-SRE Infrastructure platforms, including CloudFormation alarm provisioning and Infrastructure-as-Code delivery (IISS-487).
- **Inventory API Development:** Building the `ia-it-sre-infrastructure-inventory` Flask REST API for programmatic discovery of all deployed AWS services across the IA-IT-SRE Infrastructure, with a discoverer registry pattern that provides a live, per-platform inventory of all AWS services.
- **Cloud Infrastructure Engineering:** Authoring parameterized CloudFormation and AWS SAM templates, managing IAM roles for cross-account Grafana access, enabling CloudWatch Container Insights on ECS clusters, and navigating AWS service constraints encountered during delivery.
- **Scrum and Meeting Facilitation:** Leading daily standup calls on a rotation basis, covering progress updates, blockers, and coordination across the team. Attending and contributing to a range of recurring meetings including dashboard design reviews (with Kri and Horace),

knowledge-transfer sessions (RPA platforms, CitcoWorks, Event Store, AExeo), security and compliance reviews with internal audit teams, and ad-hoc stakeholder syncs with Kishor and the wider CTM team.

- **JIRA Story Creation and Management:** Translating verbal requirements and task descriptions communicated by Kishor during meetings or standups into structured JIRA stories and tasks with clear acceptance criteria, subtask decomposition, and relevant links. Also raised defect tickets for issues identified through root cause analysis of production incidents, assigning them to the appropriate development or platform teams for resolution.
- **RPA Production Support:** Providing operational support for Blue Prism and UiPath automation processes during dedicated support rotation weeks (11 AM – 7 PM EST shift, once every four weeks). Responsibilities included monitoring running bot queues, triaging and resolving service desk tickets from business and platform teams across Citco, onboarding new clients for Corporate Actions workflows, retriggering failed processes (MESO, VPL, FX Closeouts), and escalating to the senior team or development leads where required.
- **Incident Response and Hotfix Engineering:** Investigating and resolving production incidents including the CAIS Deadline credential failure (IISS-706), MESO month-end stuck tasks (IISS-741), and VPL pricing failures (IISS-753, IISS-876) through root cause analysis and targeted code or configuration fixes.
- **AI-Augmented Engineering:** Leveraging the Kiro IDE, Amazon Q, and Claude Sonnet for spec-driven development, automated code scaffolding, CloudFormation template generation, and test suite creation, integrating AI-native tooling into the day-to-day engineering workflow.
- **Technical Documentation:** Maintaining Confluence knowledge-base pages for Lambda function inventories, runbooks, incident post-mortems, and architecture decisions; contributing to JIRA story decomposition and time-log tracking across all deliverables.

These responsibilities required a blend of cloud engineering expertise, operational discipline, and cross-functional communication skills to deliver production-grade monitoring infrastructure while maintaining the reliability of live financial automation processes.

Chapter 2. Executive Summary

This report documents my Mitacs Business Strategy Internship (BSI) at Citco Technology Management (CTM), Halifax, Nova Scotia, conducted between **Mar 15 and Aug 31, 2026** under the project title “*Intelligent Automation and Reliability Engineering for Financial Services*” (Mitacs reference IT49539).

Prior to my joining, the IA-IT-SRE team had no centralized observability platform for the IA-IT-SRE Infrastructure — a collection of six platforms (automationhub, bpm, citcoworks, ia-bpo, meridian, rpacfs1) spanning Lambda functions, ECS clusters, EC2 instances, MSK Kafka pipelines, and a range of database and messaging AWS services. Two documented production failures — a 29-day silent outage of the CAIS-Pricing GenAI pipeline (IISS-430) and a 7.5-hour Meridian event processing delay (IISS-472) — both went undetected until business teams escalated them, exposing the team’s reliance on reactive, manual incident detection. The research question framed for this engagement was: *can AWS-native services achieve Dynatrace-equivalent observability at a fraction of the cost?*

My approach was incremental across three phases. I first built CloudWatch dashboards targeting Lambda functions — the AWS service at the centre of the silent outage — and documented each function’s trigger type and runtime across all six platforms (IISS-435, IISS-455, IISS-461, IISS-469). I then developed a registry-based IA Infrastructure inventory API that queries the AWS API dynamically and returns live, per-platform lists of all deployed AWS services (IISS-482–486, IISS-507). Finally, I built a centralized Amazon Managed Grafana (AMG) workspace as a proof-of-concept, covering all 13 AWS service categories across all six platforms via dynamic platform template variables (IISS-487). The RPA production support rotations ran in parallel throughout the internship.

The key outcomes of this work are:

- Kafka consumer lag detection reduced from a 3.5-hour business-team escalation window to **under 30 seconds** via a CloudWatch MSK EstimatedMaxTimeLag alarm, directly addressing the the Meridian Incident failure mode
- Dynamic CloudWatch alarms deployed across all monitored AWS services on all six platforms, where none or limited existed before
- Grafana AMG workspace with **14 rows** covering every AWS service category in the IA-IT-SRE Infrastructure

- AWS service inventory across all six platforms, with **99% automated test coverage** on the inventory API
- AWS-native stack achieves **~60% Dynatrace feature parity** at \$77/month (CloudWatch + Grafana AMG + X-Ray), rising to **~80%** with Application Signals

The chapters that follow detail the tools and technologies used, the full project context and JIRA scope, the SRE methodology and requirements, the phased technical implementation, and the broader learning outcomes of the engagement.

Chapter 3. Tools & Technologies

During my internship, the following tools, AWS services, and platforms were used to build the IA-IT-SRE Infrastructure monitoring deliverables, develop the inventory API, and support RPA production operations.

3.1 AWS Cloud Services

- **Amazon CloudWatch** — AWS's primary observability service, used for collecting metrics, logs, and alarms across all monitored AWS services. Metrics Insights SQL and Logs Insights were used for dashboard queries and log analysis.
- **AWS CloudFormation / SAM** — Infrastructure-as-Code framework used to provision CloudWatch alarms, Grafana workspaces, and Lambda functions via parameterized YAML templates.
- **AWS Lambda** — Serverless compute service hosting the automation workloads across all six platforms, and used as the runtime for the CloudFormation alarm provisioner and the resource inventory API.
- **Amazon ECS (Elastic Container Service)** — Container orchestration service used to run the Meridian platform services. CloudWatch Container Insights was enabled to surface task-level CPU and memory telemetry.
- **Amazon MSK (Managed Streaming for Apache Kafka)** — Managed Kafka service used as

the Meridian event pipeline backbone. Consumer lag metrics from MSK were the primary observable for the Meridian Incident post-mortem monitoring gap.

- **Databricks** — Unified data analytics platform used within the Meridian event pipeline for ingesting and processing automated system events from Citco Recs. Events are received and processed in Databricks before being consumed downstream by the OpenSearch layer.
- **Amazon SQS (Simple Queue Service)** — Message queue service used as an inter-service trigger mechanism for Lambda functions. Queue depth and oldest message age were monitored as leading indicators of processing backlogs.
- **Amazon EC2** — Virtual machine service hosting RPA bot nodes and supporting infrastructure across the platforms.
- **Amazon EKS (Elastic Kubernetes Service)** — Managed Kubernetes service used by the AutomationHub platform for container workloads.
- **Amazon ECR (Elastic Container Registry)** — Container image registry used to store and version Docker images for ECS and EKS deployments.
- **Amazon OpenSearch Service** — Search and analytics engine used by the Meridian platform as the event indexing layer queried by downstream services.
- **Amazon RDS Aurora** — Relational database service used by Meridian and other platforms for structured data storage.
- **Amazon DocumentDB** — MongoDB-compatible document database used by Meridian for document-oriented workloads.
- **Amazon Neptune** — Graph database used by the CitcoWorks platform for the Knowledge Graph hierarchy.
- **Amazon ElastiCache (Redis/Valkey)** — In-memory caching service used by Meridian's ODL layer for fast event state access.
- **Amazon S3** — Object storage used for CloudFormation template staging, Lambda deployment packages, and automation output artefacts.
- **AWS Secrets Manager** — Credential management service used to store and rotate API keys,

database passwords, and service account tokens across all platforms.

- **AWS IAM** — Identity and access management service used to define least-privilege roles for the Grafana data source, Lambda provisioner, and inventory API.
- **AWS CodeCommit** — Git-based source control service used to host all automation and infrastructure repositories.

3.2 Observability & Monitoring

- **Amazon Managed Grafana (AMG)** — Fully managed Grafana service used to build the centralized IA-IT-SRE Infrastructure dashboard with 14 rows covering all monitored AWS services across all six platforms.
- **CloudWatch Container Insights** — Managed telemetry extension for ECS and EKS that surfaces task-level CPU, memory, and network metrics, enabled on the Meridian clusters.
- **AWS X-Ray** — Distributed tracing service researched and partially evaluated for Lambda and ECS instrumentation; full deployment scoped as future work.
- **AWS Application Signals** — AWS-native SLO management and service health monitoring service evaluated as part of the Dynatrace parity analysis; identified as the path from 60% to ~80% AWS-native feature parity.
- **Dynatrace** — Enterprise APM platform already in use by the team for Blue Prism automation observability, used as the functional benchmark for the AWS-native observability PoC.

3.3 Development Tools

- **Python 3** — Primary programming language used for all automation scripts, the inventory API, and CloudFormation provisioner logic.
- **boto3** — AWS SDK for Python, used to interact with all AWS services programmatically in the inventory API and alarm provisioner.
- **Flask (async)** — Lightweight Python web framework used to build the ia-it-sre-infrastructure-inventory REST API with async route handlers.
- **FastAPI** — Python API framework used in the initial inventory API prototype before being

migrated to Flask.

- **Pydantic** — Data validation library used for type-safe response models in the inventory API.
- **pytest + Hypothesis** — Testing frameworks used to build the automated test suite for the inventory API, achieving 99% code coverage with unit, integration, and property-based tests.
- **TypeScript / ReactJS / MUI** — Frontend technology stack used for the inventory dashboard UI (IISS-509), with ReactJS for component-based UI, MUI (Material UI) for design-system components, and TypeScript for type-safe frontend development.
- **Postman** — API testing tool used to validate and document the inventory REST API endpoints during development.
- **DBeaver** — Universal database client used to inspect and query RDS, DocumentDB, and other database services across the platforms during incident investigation and development.
- **PowerShell / Shell Scripts** — Scripting environments used to automate deployment tasks, run AWS CLI commands, and execute operational scripts on Windows and Linux hosts.
- **Visual Studio Code (VS Code)** — Primary code editor used for Python, TypeScript, and YAML development, extended with the Kiro IDE plugin for AI-assisted engineering.
- **JIRA** — Project tracking tool used for sprint planning, story decomposition, and time logging across all internship deliverables.
- **Confluence** — Internal wiki used for architecture documentation, Lambda function inventory pages, and runbooks.
- **Git / AWS CodeCommit / Bitbucket** — Version control system with two hosted repository services: AWS CodeCommit for Python and IaC repositories, and Bitbucket for UiPath automation repositories.
- **Remote Desktop Connection (RDC)** — Used to connect to remote Windows machines hosting RPA bot nodes and on-premises infrastructure for incident investigation and configuration changes.

3.4 AI-Native Development Tools

- **Kiro IDE** — AI-powered development environment built on VS Code, used for spec-driven development (`requirements.md` → `design.md` → `tasks.md`) and agentic code generation for the inventory API.
- **Amazon Q** — AWS integrated code assistant used for inline boto3 completions and project-level context retrieval.
- **Claude Sonnet Opus (Anthropic)** — Large language model accessed via Kiro, used for architecture reasoning, CloudFormation template drafting, and test strategy design.

3.5 RPA Platforms

- **Blue Prism** — Legacy RPA platform hosting daily, monthly, and quarterly fund operations automation processes, monitored during production support rotations.
- **UiPath Cloud / On-Premises** — Target RPA platform for new automation development and Blue Prism migrations, operated via UiPath Orchestrator with SQS-triggered job invocations.

Chapter 4. Project Overview

4.1 Project Background and the Motivating Incident

4.1.1 Pre-Internship Monitoring Posture

Before my internship began in mid-Mar 2026, the IA-SRE team operated without a centralized observability stack. Monitoring relied entirely on an *ad hoc* model: business teams in Dublin, London, and Halifax would notice anomalies in fund processing outputs and raise tickets. There were no CloudWatch alarms, no dashboards aggregating health across the six platforms, and no mechanism for the SRE team to receive proactive notification of degrading conditions — the team was blind to *saturation* and *error rate* signals until they had already caused downstream business impact.

4.1.2 CAIS-Pricing Silent Outage (Jan-Feb 2026)

Between **Jan 20 and Feb 17, 2026**, the CAIS-Pricing Extract GenAI pipeline under the `rpacfs1` platform stopped delivering output for **29 consecutive days** without a single alert firing. The root cause was gradual exhaustion of the external CTI AI model API token, which controls the async SQS callback that triggers the downstream Lambda stages; without that callback, three downstream functions dropped to zero invocations while two upstream functions kept running, making the pipeline appear healthy. The outage was detected only when business users reported missing pricing data — 29 days after onset. The post-mortem (IISS-430) identified the absence of throughput alarms, SQS queue depth monitoring, and end-to-end output verification as the root causes of the failure going unnoticed. Sub-task IISS-435 was assigned directly to me as the immediate remediation and became the starting point for the full monitoring programme.

4.1.3 The Meridian Incident (Apr 2026)

The Meridian Incident is described in full detail in Section ?? of Chapter 1. In brief: a surge of automated system events from Citco Recs overwhelmed Meridian's OpenSearch consumption layer, causing Kafka consumer lag to reach approximately **9 million messages** and halting daily statement processing for 7.5 hours. The SRE team received no alert; the Dublin business team escalated over three and a half hours after the first failure signal. The post-mortem recorded: *"No proactive monitoring/alerting in place on the SRE side for Meridian event volume anomalies."* This became the primary driver for IISS-487.

4.1.4 From Incident to Programme

Both incidents shared the same root cause: no proactive monitoring. The post-mortems established two specific gaps — no Kafka consumer lag alarm, and no centralized visibility across the IA-IT-SRE Infrastructure — that directly motivated the three work streams described in the following section. Kishor elevated IISS-487 to **Critical** priority on May 6, 2026, and my internship pivoted from exploratory dashboard research to a full-scale observability platform build.

4.2 Project Overview

The two motivating incidents established a clear engineering mandate: build proactive observability across the entire IA-IT-SRE Infrastructure. Three work streams were pursued sequentially and

in parallel, each one directly motivated by the findings and limitations of the previous. A fourth ongoing responsibility — RPA production support — ran throughout the internship period.

4.2.1 Work Stream 1: CloudWatch Dashboards and IA Infrastructure Monitoring

Following the CAIS-Pricing Silent Outage, the immediate remediation was to build a unified CloudWatch monitoring dashboard starting with Lambda — the AWS service at the centre of that incident. The goal was to surface invocation count, error rate, duration, and throttle metrics for every Lambda function across all six platforms, giving the SRE team visibility that had not existed before.

Once Lambda was covered, the scope was extended incrementally to the other AWS services in the IA-IT-SRE Infrastructure: SQS queues (queue depth and oldest message age), EC2 instances (CPU and memory via CloudWatch Agent), ECS clusters and tasks (via CloudWatch Container Insights), EKS node metrics, and database services. Each resource type was added as a separate CloudWatch dashboard, establishing the tag-based resource discovery patterns — including the AppName fall-back strategy for shared BudgetCode environments — that all later work built upon.

These CloudWatch dashboards were effective for per-service visibility, but they exposed a fundamental limitation: CloudWatch dashboards are siloed per AWS service and do not support dynamic cross-service filtering. There was no way to view Lambda, SQS, and ECS health for a single platform side by side in one workspace. This limitation directly motivated Work Stream 2. Key deliverables: IISS-435, IISS-455, IISS-461, IISS-469, IISS-482–486.

4.2.2 Work Stream 2: Grafana AMG Centralized Dashboard

Following the Meridian Incident, Kishor elevated IISS-487 to **Critical** priority. The goal was to build a proof-of-concept (PoC) centralized observability platform using **Amazon Managed Grafana (AMG)** — addressing what the per-service CloudWatch dashboards could not: a unified, cross-platform, cross-service view of the entire IA-IT-SRE Infrastructure in a single workspace.

The scope was defined by the Meridian post-mortem action items and refined through requirements sessions with Kishor: the dashboard needed to cover every AWS service already monitored in Work Stream 1 (Lambda, SQS, MSK/Kafka, EC2, ECS, EKS, ECR, RDS Aurora, DocumentDB, Redis/Valkey, Neptune DB, OpenSearch), support dynamic filtering by platform and AWS service via template variables, and include CloudWatch alarms with SNS notifications to the SRE team. The broader research objective was to evaluate whether this AWS-native stack could achieve

Dynatrace-equivalent observability — documenting the feature parity achieved, the gaps that remain, the limitations encountered, and the incremental cost at each tier.

The dashboard was built on CloudWatch as the sole data source (via the native Grafana CloudWatch plugin), using Metric Insights SQL for cross-resource aggregation and CloudWatch Logs Insights for log-derived panels. An Infrastructure-as-Code provisioner — a Lambda-backed CloudFormation pipeline — was developed to dynamically discover the AWS service fleet at deploy time and synthesize per-service alarm definitions, enabling a single deployment to cover all monitored AWS services without manual enumeration. Key limitations discovered and documented during this work stream — including the Metric Insights SQL percentile restriction, the `SEARCH()` visibility gap for idle resources, and the 64 KB Secrets Manager payload limit — are detailed in Chapter 7. Key deliverable: IISS-487 (140 hours).

4.2.3 Work Stream 3: IA Infrastructure Inventory API

A critical limitation surfaced during the Grafana dashboard build: the `WHERE` clause in CloudWatch Metric Insights SQL does not support pattern-based filtering (no wildcard or regex on dimension values). To filter a panel to a specific platform, an exact dimension value match was required — but there was no reliable way to enumerate the available values dynamically from within Grafana. The `dimension_values()` function returned partial results for large fleets, and cross-platform queries required hardcoding platform names into template variable queries.

To solve this, the Lambda discovery script that had been built for the CloudWatch dashboards was extended into a full-fledged **Flask REST API**: the `ia-it-sre-infrastructure-inventory` service. The API discovers AWS services across all six platforms, regions, and environments in real time at query time, and exposes structured resource lists via REST endpoints. A Grafana dashboard panel can call the API to retrieve the current list of Lambda functions, ECS clusters, or SQS queues for a given platform, and use the response to populate template variable options dynamically — ensuring the dashboard always reflects the live fleet without requiring manual updates.

The API uses a discoverer registry pattern: a registry maps `(resource_type, platform)` tuples to concrete discoverer classes, allowing new platforms or AWS service types to be onboarded by implementing a single class. The service was built with Python and `flask[async]`, using `boto3` for all AWS API calls and `Pydantic` for response validation. The test suite achieved 99% code coverage across unit, integration, and property-based tests. Key deliverable: IISS-507.

4.2.4 Work Stream 4: RPA Production Support

In parallel with the three engineering work streams, I participated in dedicated RPA production support rotation weeks as part of the team's **Follow the Sun** support model. The North America slot covers 11 AM – 7 PM EST, and I held this slot once every four weeks, serving as the primary point of contact for all RPA and automation-related incidents during the shift.

The support model is structured across three tiers. **Level 1** is handled by the AI/RPA Support Agent (24/7). **Level 2** is owned by the Innovation IT Support team (us) email channel, operating under a Follow the Sun model with a 1-hour acknowledgement SLA and 8-hour resolution target. **Level 3** is the SRE team (ia_it_sre@citco.com), engaged for complex production issues, root cause analysis, and service improvement, with a 1-day response SLA. As an SRE intern I operated across both L2 and L3 responsibilities during support rotations.

At the start of each shift I reviewed all open items across the relevant channels: the **SD (Service Desk) Portal** for incoming tickets, the **innovation_it_support@citco.com** email inbox for L2 escalations, the **IA_INCIDENTS@citco.com** distribution list for incident notifications, and **Microsoft Teams** messages and channel notifications from business teams and the outgoing shift. I also reviewed the Confluence shift handover log left by the Manila (Phillipinse) shift to pick up any carried-over items or approaching SLA breaches. Ticket ownership rules required that whoever acknowledged a ticket owned it to resolution — handover notes had to include actions taken, pending items, blockers, and clear next steps for the incoming shift.

Incident work followed the full lifecycle. I monitored running Blue Prism and UiPath bot queues (particularly the VPL queue opening at 5 PM EST daily), triaged service desk tickets, and for each issue performed root cause analysis: investigating CloudWatch logs, UiPath Orchestrator heartbeat status, Blue Prism control room logs, and application-level error messages. Where a fix was within support scope — updating a Corporate Actions FileMask for client onboarding, retriggering a stuck MESO pipeline, correcting a configuration file — I resolved it directly. Where the root cause required a code change beyond support scope, I prepared a documented RCA, proposed a resolution approach, and raised or assigned a JIRA to the appropriate development team, copying ia_it_sre@citco.com per the incident escalation procedure.

Notable incidents handled during support rotations included the CAIS Deadline credential failure (IISS-706), where I identified the Secrets Manager rotation gap and delivered a fail-fast hotfix; the MESO month-end stuck tasks (IISS-741, Critical) requiring manual pipeline retrigger; and the VPL

Pricing GTL incorrect booking (IISS-876), where I performed the RCA and coordinated the fix with the business team.

Chapter 5. Project Goals & Deliverables

I set the following engineering objectives at the outset of the internship, guided by the two motivating incidents and the requirements sessions with Kishor and Kri. Each objective maps directly to one of the four work streams described in Chapter 4.

5.1 Project Goals

The primary goals of the internship project were as follows:

- **Eliminate reactive-only incident detection.** Replace the dependency on business-team escalation as the primary detection mechanism with proactive CloudWatch alarms that fire within seconds of a threshold being crossed, directly addressing the failure modes exposed by the CAIS-Pricing Silent Outage and the Meridian Incident.
- **Build a centralized observability platform for the IA-IT-SRE Infrastructure.** Provide an eagle-eye view across all six platforms and all 13 AWS service categories, enabling the SRE team to assess platform health, drill down to individual AWS services, and respond to incidents without navigating six separate CloudWatch consoles.
- **Automate AWS service discovery and inventory.** Replace manual Confluence-maintained Lambda lists with a programmatic, always-current inventory API that enumerates all deployed AWS services per platform and exposes them via REST endpoints for use by the dashboard and other downstream tooling.
- **Evaluate AWS-native observability as a Dynatrace alternative.** Conduct a structured feature-parity analysis comparing the AWS-native stack (CloudWatch, Grafana AMG, X-Ray, Application Signals) against Dynatrace capabilities, quantifying the achievable parity percentage and the incremental monthly cost at each tier.
- **Deliver Infrastructure-as-Code for all monitoring components.** Ensure all alarms, dash-

boards, IAM roles, and supporting infrastructure are provisioned via parameterized CloudFormation templates and AWS SAM, making the platform reproducible, version-controlled, and environment-portable.

- **Establish engineering quality standards.** Produce a test suite for the inventory API with 99% code coverage, document all architectural decisions and limitations, and maintain a Confluence knowledge base for the Lambda inventory, runbooks, and incident post-mortems.

5.2 Deliverables and Technical Scope

This section documents the technical scope and stack for each primary deliverable. Detailed implementation decisions are covered in Chapter 9.

Table 5.1: Project Deliverables — JIRA Scope

JIRA	Title / Scope
IISS-435, 455, 461, 469	CloudWatch Lambda Monitoring Dashboard and inventory
IISS-482-486	IA Infrastructure Monitoring Platform (CloudWatch dashboards extended to all platforms)
IISS-487	Centralized Grafana AMG SRE Dashboard (primary deliverable)
IISS-507-510	IA Infrastructure Inventory API (Flask REST API)
IISS-706, 538, 741, 753, 876	RPA Production Support Incidents

5.2.1 IA Infrastructure Monitoring Platform (IISS-435, 455, 461, 469, 482-486)

Platform: Native Amazon CloudWatch dashboards built with Python-based dashboard builders and CloudFormation.

Scope: Initially scoped to the `rpacfs1` and `ia-bpo` platforms (Lambda functions, Confluence inventory), then extended to all six platforms — `automationhub`, `bpm`, `citcoworks`, `ia-bpo`, `meridian`, and `rpacfs1` — covering Lambda, EC2, ECS, EKS, SQS, and supporting AWS services. Each dashboard uses `BudgetCode` as the primary tag filter and `AppName` as the mandatory fallback discriminator for `meridian` and `citcoworks` platforms.

SQS Registry Architecture: A three-tier fallback model for resilient SQS queue data retrieval: live CloudWatch API, S3-cached snapshot on API failure, and locally bundled queue inventory as the final fallback.

Key limitation: CloudWatch Metric Insights SQL does not support pattern matching on dimension values, making dynamic resource enumeration for Grafana template variables impractical — this directly motivated the Inventory API (WS3).

5.2.2 Centralized Grafana SRE Dashboard (IISS-487)

Platform: Amazon Managed Grafana (AMG) v12.4, with Amazon CloudWatch as the sole data source via the native Grafana CloudWatch plugin.

Data layer: CloudWatch Metric Insights SQL used for all multi-resource panels — a single query returns per-resource time series across the full fleet. CloudWatch Logs Insights used for log-derived error panels with structured JSON field extraction.

Dashboard structure: 14 rows — two aggregate overview rows (Health Overview and Performance Overview) followed by 12 per-service rows: SQS, Lambda, EC2, ECS Infrastructure, ECS APIs, MSK/Kafka, DocumentDB, RDS/Aurora, Redis/Valkey, Neptune DB, OpenSearch, and Alerts & Incidents. Cross-service filtering via Grafana template variables allows operators to scope the entire workspace to a single platform or AWS service via dropdown selection.

Alerting layer: CloudWatch alarms provisioned across all monitored AWS services, each wired to an SNS topic routing to the SRE team. Composite alarms aggregate per-service states into platform-level health signals.

Infrastructure-as-Code (IaC): AWS SAM with parameterized CloudFormation YAML templates (one template per AWS service category). A Lambda-backed CloudFormation provisioner dynamically discovers the current fleet at deploy time and synthesizes alarm resource definitions, deployed across four stacks: Lambda, ECS, SQS, and MSK alarms. Templates are staged in S3 (migrated from Secrets Manager due to the 64 KB payload limit) and fetched by the provisioner Lambda at runtime.

IAM model: 16-role cross-account agent permission model granting the Grafana service account read-only access to CloudWatch metrics, logs, and EC2 tags across all IA AWS accounts.

Container Insights: Enabled on five of the six Meridian ECS clusters, providing task-level CPU, memory, network I/O, and storage telemetry surfaced in the ECS dashboard rows.

5.2.3 IA Infrastructure Inventory API (IISS-507–510)

Stack: Python 3, flask[async], boto3, Pydantic, serverless-wsgi (Lambda deployment), pytest and Hypothesis (property-based testing).

Discoverer Registry Pattern: A central registry maps (resource_type, platform) tuples to concrete discoverer classes. New platforms or AWS service types are onboarded by implementing a single class, with zero changes to routing, serialization, or test infrastructure. Route handlers dispatch discoverers concurrently via `asyncio.gather()`.

Tag disambiguation: The meridian and citcoworks platforms share the same BudgetCode tag value. A mandatory two-stage filter — primary BudgetCode match, secondary AppName discriminator — separates resources from the two platforms across all discoverers.

API: REST endpoints for Lambda, ECS, EC2, SQS, S3, and health, each supporting an optional `?platform=` filter. Each resource response carries name, type, platform, tags, and ARN.

Test suite: 99% code coverage across unit, integration, and property-based tests.

5.3 Acceptance Criteria

The following criteria defined whether each deliverable met its stated objectives.

5.3.1 IA Infrastructure Monitoring Platform

- **Platform coverage:** CloudWatch dashboards exist for all six platforms covering the key AWS services (Lambda, ECS, EC2, SQS, EKS) with tag-based resource filtering.
- **Resilient SQS monitoring:** The SQS registry pipeline remains functional during AWS service degradation via the three-tier fallback model.

5.3.2 SRE Observability Platform

- **Proactive incident detection:** The team can be alerted to Kafka consumer lag, Lambda error rate spikes, ECS CPU saturation, and SQS queue backlogs before business teams escalate — directly addressing the failure modes exposed by the CAIS-Pricing Silent Outage and the Meridian Incident.

- **Cross-platform visibility:** The SRE team can assess the health of all six platforms and all monitored AWS services from a single Grafana workspace without navigating individual CloudWatch consoles.
- **Alarm coverage:** CloudWatch alarms are deployed and active across all monitored AWS services on all six platforms, with SNS notifications routed to the SRE team.
- **Container-level telemetry:** Task-level CPU and memory metrics are available for the Meridian ECS clusters, enabling resource saturation to be detected before service degradation occurs.
- **Reproducible infrastructure:** All monitoring infrastructure — Grafana workspace, alarms, IAM roles, S3 storage — is provisioned as Infrastructure-as-Code and can be redeployed consistently across environments.

5.3.3 IA Infrastructure Inventory API

- **Dynamic discovery:** The API returns a current, accurate list of deployed AWS services per platform at query time, enabling Grafana template variables to reflect the live fleet without manual updates.
- **Tag disambiguation:** Resources from the meridian and citcoworks platforms are correctly separated despite sharing the same BudgetCode tag value.
- **Extensibility:** New platforms or AWS service types can be onboarded by implementing a single discoverer class with no changes to the API routing or test infrastructure.
- **Test coverage:** 99% automated test coverage across unit, integration, and property-based tests.

5.3.4 RPA Production Support

- **Incident response:** Production incidents are triaged, root-caused, and either resolved within the support shift or escalated with a documented RCA and proposed fix.
- **SLA compliance:** Tickets are acknowledged within the L2 response SLA and carried over with complete handover notes per the IA Support Model.

Chapter 6. Learning Goals

The following learning goals were established at the start of my internship in consultation with my academic and industry supervisors. They are organized into technical and non-technical categories to reflect the dual competency development expected of an MCDA industry placement.

6.1 Technical Learning Goals

- **AWS Cloud Services and Infrastructure.** Gain production-level proficiency with the AWS services central to the IA-IT-SRE Infrastructure. During the part-time onboarding phase, I focused on understanding the full AWS service landscape in use at Citco: Amazon CloudWatch (metrics, alarms, Logs Insights, Metric Insights SQL), AWS Lambda (serverless execution, cold-start behaviour, concurrency), Amazon ECS and EKS (containerized workloads), Amazon EC2 (virtual machine infrastructure), Amazon MSK (managed Kafka, consumer lag), Amazon SQS (queue-based messaging), Amazon SNS (alerting), Amazon S3, AWS CloudFormation and SAM (Infrastructure-as-Code), AWS CodePipeline (CI/CD), and AWS Secrets Manager (credential management). A deliberate goal was exposure to the full spectrum of database categories deployed at Citco: Amazon DynamoDB (key-value), Amazon DocumentDB (document), Amazon Neptune (graph database — CitcoWorks Knowledge Graph), Amazon RDS Aurora (relational), Amazon ElastiCache for Redis and Valkey (in-memory caching — Meridian ODL tier), and Amazon OpenSearch Service (search and analytics). Additional infrastructure covered includes Amazon ECR, AWS CodeCommit, and the AWS IAM permission model (including cross-account role chaining for Grafana data source access).
- **Observability, Monitoring, and the Four Golden Signals.** Develop a working understanding of modern observability principles and apply them to a production financial services platform. This goal was developed through discussions with Kri (SRE mentor) on the four golden signals framework — latency, traffic, errors, and saturation — and their application to the IA platform AWS service fleet. Full details are covered in Chapter 7.

Beyond the four golden signals, learning goals included understanding key observability metrics for each AWS service on the dashboard — MSK consumer lag, SQS queue depth and oldest message age, ECS task CPU and memory, Lambda duration and concurrent executions, EC2 CPU and network, OpenSearch JVM heap pressure, and Redis memory utilization — as well as evaluating advanced observability tooling including AWS X-Ray (distributed tracing),

AWS Application Signals (SLO management), Grafana Trace Maps, and improvement opportunities for the existing IA ML/LLM RPA Support Agent.

- **Infrastructure-as-Code and CI/CD.** Author parameterized CloudFormation and AWS SAM templates for alarm and infrastructure deployment; develop a Lambda-backed dynamic alarm provisioner; and understand the AWS CodePipeline CI/CD model used for automation deployments across the platforms.
- **Grafana AMG Platform.** Build and operate a production-grade Amazon Managed Grafana workspace: configure CloudWatch as a data source, design collapsible dashboard rows for each AWS service, implement platform template variables for dynamic filtering, and learn Grafana panel types (time series, stat, bar gauge, heatmap, top-K list, table with deep-link URLs).
- **Backend API Development.** Write production-quality Python APIs using Flask and FastAPI with async route handlers, boto3 for AWS service interaction, Pydantic for response validation, and pytest with Hypothesis for property-based testing.
- **Software Engineering Practices.** Practice spec-driven development via the Kiro IDE workflow (.kiro/specs/ with requirements.md, design.md, tasks.md), and apply AI-augmented engineering using Kiro (Claude Sonnet) and Amazon Q to accelerate design documentation, code scaffolding, and test generation.
- **Proactive vs. Reactive Reliability Engineering.** Transition from a reactive support model (incident detection by business escalation) to a proactive one (alarm-backed detection within 30 seconds) by applying SLI/SLO targets and error budget concepts to real production workloads.

6.2 Non-Technical Learning Goals

- **Scrum and Engineering Leadership.** Lead daily standup calls on rotation, facilitate team coordination, translate verbal requirements from Kishor into structured JIRA stories with clear acceptance criteria and subtask decomposition, and develop task estimation and prioritisation skills in a production engineering environment.
- **Cross-Functional and Cross-Timezone Collaboration.** Build working relationships across Halifax IA-SRE, Innovation IT Hyderabad (IST), Dublin and London business operations teams,

and the RPA support tier. Participate in structured knowledge-transfer sessions, design reviews, security and compliance meetings, and stakeholder syncs across time zones.

- **Production Incident Response and RCA.** Develop the discipline of structured root cause analysis: follow the IA Support Model escalation tiers, document findings in SD tickets and Confluence, communicate clearly with business and platform teams under time pressure, and propose fixes or raise JIRA defects with actionable resolution steps.
- **Technical Communication and Documentation.** Produce engineering-grade written deliverables: incident RCA reports, architecture design documents, Confluence knowledge-base pages, runbooks, and this academic report.
- **Financial Domain Knowledge.** Develop working familiarity with fund administration operations — VPL queue (5 PM EST daily deadline), Corporate Actions workflows (Mandatory, Voluntary, Ops Transfer), MESO month-end cycles, and CAIS regulatory filing deadlines — to make informed monitoring and incident response decisions.

Chapter 7. Methodologies & Requirements

During my internship I adopted Site Reliability Engineering as the primary methodology, applying the four golden signals framework and SLI/SLO discipline to the observability gap exposed by the CAIS-Pricing Silent Outage and the Meridian Incident. This chapter covers the SRE principles I operationalised, the gap analysis that established the starting posture, and the Dynatrace parity evaluation that framed the research question.

7.1 SRE Principles

Site Reliability Engineering is an operational discipline that applies software engineering thinking to the problem of keeping production systems reliable at scale. Its conceptual vocabulary — golden signals, Service Level Indicators (SLIs), Service Level Objectives (SLOs), and error budgets — provided the theoretical backbone for every design decision I made during this internship.

7.1.1 The Four Golden Signals

I discussed the four golden signals framework with Kri during a dedicated knowledge-transfer session. They define the minimum instrumentation required to characterise the health of any service, regardless of the underlying technology.

1. **Latency** — *How long does a request take?* For the IA platform, latency manifests as Lambda function duration (measured at the P50, P95, and P99 percentiles), and as ECS service response time for API-facing containers. Sustained increases in P99 latency are an early warning of resource saturation or dependency degradation before errors appear.
2. **Traffic** — *How much demand is on the system?* Traffic is measured as Lambda invocation count per unit time, SQS message ingestion rate for queue-backed automations, and MSK throughput (bytes in/out per second) for the Meridian Kafka pipeline. Traffic anomalies — sudden spikes or unexpected drops — often precede error and saturation signals.
3. **Errors** — *What is the rate of failed requests?* The IA platform tracks errors as Lambda error and throttle counts, ECS task failure counts, and `NumberOfMessagesFailed` on SQS queues. An error rate that crosses a defined threshold indicates a code or infrastructure defect requiring immediate attention.
4. **Saturation** — *How full is the service?* Saturation is the signal most relevant to the Apr 17, 2026 Meridian incident. It is measured as Lambda concurrent execution count relative to the account concurrency limit, ECS task CPU and memory utilisation, MSK consumer lag (the number of messages awaiting processing on a Kafka topic), and SQS queue depth with oldest-message age. A service approaching saturation degrades before it fails — making this the most actionable of the four signals.

7.1.2 Service Level Indicators and Objectives

A **Service Level Indicator (SLI)** is the specific metric chosen to represent one of the four golden signals for a service — the measurement. A **Service Level Objective (SLO)** is the target threshold set on that measurement — the acceptable limit.

For the IA platform, I selected the following SLIs and set corresponding SLO thresholds, implemented as CloudWatch alarm trigger values:

- **Lambda error rate:** $SLI = \text{Errors} / \text{Invocations per function}$. SLO threshold = fewer than

1% of invocations result in an error over any 24-hour window. Implemented as a CloudWatch alarm with Metric Math expression $\text{ErrorRate} = \text{Errors} / \text{Invocations}$.

- **MSK consumer lag:** SLI = `EstimatedMaxTimeLag` per Meridian consumer group. SLO threshold = lag must remain below **10,000 messages** at any point. Derived from the Apr 17 post-mortem: lag reached 9 million messages before detection; a 10,000-message threshold would have fired within minutes of onset.
- **ECS CPU utilisation:** SLI = task-level CPU utilisation per cluster via Container Insights. SLO threshold = no service sustains above **80%** over a 5-minute evaluation period.
- **SQS queue depth:** SLI = `ApproximateNumberOfMessagesVisible` per queue. SLO threshold = configurable per-queue maximum based on expected processing throughput.

These thresholds were encoded directly as CloudWatch alarm trigger values — each alarm fires when its SLI crosses the SLO boundary and routes an SNS notification to the SRE team. This constitutes **SLO-aligned alerting**: the alarms enforce the objectives but do not compute running error budget consumption or burn rate.

7.1.3 Error Budgets

An **error budget** is the complement of the SLO: the permitted amount of unreliability before the objective is breached. For example, if the Lambda error rate SLO is below 1% over 24 hours, the error budget is that 1% of invocations permitted to fail.

During this internship I did not implement automated error budget tracking. The Grafana dashboard surfaces current SLI values and the CloudWatch alarms fire at SLO breach — which provides a reactive signal that the budget has been consumed, but does not calculate how quickly it is being drawn down. Full burn-rate alerting (requiring Metric Math expressions dividing rolling error counts against the rolling SLO budget) was identified as a post-internship enhancement and is scoped in Chapter 14.

7.2 Observability Gap Analysis

A prerequisite for building any monitoring platform is understanding the current state of observability. Before I wrote a single line of code for IISS-487, I performed a structured gap analysis to characterise what the IA platform could and could not see about itself.

7.2.1 Pre-Internship Monitoring Posture

At the commencement of my internship in Mar 2026, the IA-SRE team's monitoring posture exhibited the following deficiencies:

- **No CloudWatch alarms on any Meridian service.** The Meridian Kafka-backed event pipeline — the most business-critical component of the IA platform — had zero proactive alerting. No alarms existed on MSK consumer lag, ECS task failure counts, Lambda error rates, or SQS queue depth for any Meridian-owned resource.
- **No centralized dashboard.** Monitoring was entirely fragmented. Operators who wished to check the health of a service were required to navigate to that service's individual CloudWatch console page. There was no eagle-eye view from which to assess the overall health of the platform.
- **Incomplete Container Insights coverage.** Of the six Meridian ECS clusters, only one (enabled in the DEV environment since Jul 2024) had Amazon CloudWatch Container Insights activated. The remaining five clusters were invisible to task-level CPU and memory telemetry.
- **No MSK consumer lag monitoring.** Despite Kafka consumer lag being the primary leading indicator of Meridian processing health — as demonstrated conclusively by the Meridian Incident — no metric or alarm existed for this signal. The `EstimatedMaxTimeLag` and `SumOffsetLag` metrics available natively from Amazon MSK were not surfaced anywhere.
- **No SLO tracking or error budget calculation.** The team had no defined SLIs, no published SLO targets, and therefore no basis for calculating error budgets or determining when the platform was in an unacceptable reliability state.
- **No automated resource inventory.** The inventory of Lambda functions, ECS clusters, and EC2 instances was maintained manually via Confluence pages and spreadsheets. This made it impossible to know, at any given moment, exactly which resources existed across all six platforms.
- **RPA bot monitoring limited to orchestrator UIs.** Blue Prism and UiPath bots were monitored exclusively through their respective orchestrator platforms (Blue Prism Orchestrator and UiPath Cloud). No CloudWatch integration existed, and there was no mechanism for the SRE team to receive proactive notification of bot failures or queue backlogs outside the

orchestrator UI.

- **Reactive operational posture.** The cumulative effect of the above gaps was that the team's primary incident detection mechanism was business-team escalation. When a fund processing automation failed silently, the first signal received by the SRE team was a ticket or message from operations staff in Dublin, London, or Halifax who had noticed anomalous fund processing outputs. This reactive posture introduced structural latency between failure onset and engineering response.

7.2.2 Gap Analysis Summary

Table 7.1 summarises the pre-internship gaps and the post-internship resolutions for each monitoring domain.

Table 7.1: Observability Gap Analysis — Pre- vs. Post-Internship

Domain	Pre-Internship State	Post-Internship State
CloudWatch Alarms	0 alarms on any IA resource	38 alarms across 7 AWS service categories
Centralized Dashboard	None	14 rows, 110+ panels (Grafana AMG)
Container Insights	1 of 6 Meridian clusters	5 of 6 Meridian clusters
MSK Consumer Lag	No monitoring	CloudWatch alarm on EstimatedMaxTimeLag (<30 sec detection)
SLO Tracking	None	SLI/SLO targets defined; dashboard exposes current values
Resource Inventory	Manual (Confluence)	Automated REST API (Lambda, ECS, EC2 across all platforms)
Incident Detection	Business team escalation	Proactive alarm with SNS routing

7.3 Dynatrace Parity Analysis

A key framing question for my internship — introduced in Chapter 2 and developed here in methodological detail — is how closely an AWS-native observability stack can replicate the capabilities of Dynatrace, and at what incremental cost. The PoC investigation evaluated 25 Dynatrace features against five AWS tool combinations to quantify achievable parity.

Table 7.2 summarises the parity analysis for the ten most operationally relevant Dynatrace capabilities to the IA platform.

Table 7.2: Dynatrace Capability Parity — AWS-Native Stack vs Dynatrace (selected features)

Dynatrace Feature	CW	+Grafana	+X-Ray	+App Signals	+Tempo
Alerting	✓	✓	—	—	—
Infrastructure Metrics	✓	✓	—	—	—
Log Analytics	✓	✓	—	—	—
Container Monitoring	✓	✓	—	✓	—
Error Rate Tracking	✓	✓	✓	✓	—
Latency Percentiles	—	✓	✓	✓	✓
Distributed Trace Waterfall	—	—	✓	—	✓
Service Map / Topology	—	—	✓	✓	—
SLO Management	—	—	—	✓	—
Dependency Mapping	—	—	✓	✓	—
Parity (% of 25 features)	30%	33%	60%	80%	83%
Monthly Cost (est.)	\$49	\$68	\$77	\$232	\$269

Three capabilities remain **permanently outside the AWS-native reach**: Davis AI causal analysis (proprietary ML, no AWS equivalent); automatic baselining (Dynatrace learns per-service baselines over weeks; AWS alarms require manual thresholds); and session replay (full DOM capture, not available in any OSS tool). These account for approximately 9–10% of the functional gap that no combination of AWS tools can close.

Adopted stack. The PoC I delivered in this internship corresponds to the **CloudWatch + Grafana + X-Ray** column: 60% parity at approximately \$77/month. Application Signals (the path to 80% parity) requires deploying an ADOT sidecar on each ECS service and is scoped as post-internship work. The gap-to-tooling mapping is used throughout Chapter 9 and Chapter 12 to contextualise what the PoC does and does not cover.

7.4 Limitations and Mitigations

The CloudWatch and AMG constraints encountered during the build — including the SEARCH() visibility gap, Metric Insights SQL percentile restriction, IAM subscription filter boundary, Secrets

Manager payload limit, and shared BudgetCode tag ambiguity — are documented in detail in Chapter 13.

Chapter 8. Requirements Elicitation

During my internship I worked with a small set of internal stakeholders whose needs shaped the functional and non-functional requirements of the IA-IT-SRE Infrastructure monitoring platform.

8.1 Stakeholder Identification

Table 8.1 enumerates the key stakeholders, their roles, and their specific interests in the project.

Table 8.1: Stakeholder Identification

Stakeholder	Role	Interest / Influence
Kishor	Manager, CTM; Industry Supervisor	Primary sponsor; set engineering priorities; escalated IISS-487 to Critical; final acceptance authority
Horace	IT Senior Developer, RPA Platform Lead	RPA subject matter expert; delivered Blue Prism and UiPath KT; defined RPA monitoring scope
Kri	IT Developer, SRE Engineer	Delivered golden signals and alarm threshold training; reviewed dashboard design
IA IT SRE Team	Internal SRE consumers	Primary users of the Grafana dashboard and CloudWatch alarm notifications
Meridian Team	Kafka/ECS architecture owners	Provided MSK consumer group structure, ECS cluster topology, and OpenSearch integration context
Sridhar & Soundarya	Co-interns	Parallel development of related monitoring features; design review and test validation

8.2 Requirements Elicitation

The elicitation methods used were appropriate for an in-house engineering context: structured knowledge-transfer sessions, incident post-mortems, and JIRA story refinement, rather than formal user interviews or focus groups.

8.2.1 Elicitation Methods

Knowledge-Transfer Sessions (Training Week, Apr 13–18, 2026). The first week of full-time engagement was structured as an intensive knowledge-transfer programme covering the full breadth of the IA platform. Sessions with Horace on the Blue Prism and UiPath architectures, with Bhanu Teja Murari on CitcoWorks and the Event Store platform, and with the Meridian team on the Kafka event pipeline (comprising the Event Routing Service, Action Processor, and OpenSearch consumer) provided the architectural grounding needed to define meaningful SLIs. Key requirements insight from training week: the Aexo Treasury platform (cloud-based wire approval and fund movement) and the MESO platform (high-memory autoscale issue, IISS-538) each had distinct resource profiles requiring separate monitoring strategies.

Incident Post-Mortem (Apr 17, 2026). The Meridian production incident yielded the single most important requirements insight of the entire engagement: MSK consumer lag is the primary leading indicator of Meridian health. The post-mortem made concrete the latency between signal onset and detection under the reactive model, and directly specified the target for the lag alarm evaluation period (NFR2: ≤ 1 minute).

Golden Signals Session (May 25, 2026). Kri's dedicated SRE methodology session translated the theoretical four golden signals into concrete metric choices for the IA platform. This session produced the initial SLI/SLO target table and established the design constraint that all metrics must be queryable through Amazon CloudWatch — ruling out external time-series databases such as Prometheus and anchoring the architecture to CloudWatch Metric Insights as the sole data source.

JIRA Story Refinement. IISS-487 went through multiple refinement cycles between May 6 and May 11 (infrastructure deployment date). These sessions with Kishor and Kri surfaced the infrastructure-as-code requirement (FR7), the multi-environment reuse requirement (NFR5), and the credential management constraint (NFR7). The elevation of IISS-487 to Critical priority on May 6 also functionally set NFR1 (dashboard refresh latency ≤ 30 seconds) as a hard constraint rather than an aspirational target.

8.2.2 Key Insights

Three insights from the elicitation process materially shaped the resulting design:

1. **Tag-based resource discovery is non-trivial at Citco scale.** The training week revealed that the citcoworks and meridian environments share the **Shared Cloud Artifacts** BudgetCode tag, meaning a simple BudgetCode lookup conflates resources from two operationally distinct environments. This directly motivated FR9 (tag-based discovery with AppName fall-back).
2. **The primary audience for the dashboard is the SRE team, not business operators.** Business operations teams would continue to use their existing domain-specific tools; the dashboard needed to be useful for the SRE first-responder, not designed as a business reporting tool. This shaped the panel selection (technical metrics over business KPIs) and the deep-link design (panels link directly to the relevant AWS Console page).
3. **Infrastructure-as-Code is a first-class requirement, not an after-thought.** The team had experienced the cost of manually provisioned monitoring infrastructure in previous projects (configuration drift, undocumented alarms). The JIRA refinement sessions made FR7 an explicit acceptance criterion rather than a **nice to have**.

8.3 Functional and Non-Functional Requirements

The following requirements were derived from the elicitation activities described above. They governed the design and acceptance criteria for all three work streams of my internship.

8.3.1 Functional Requirements

Table 8.2 enumerates the nine functional requirements, each with the primary deliverable that satisfies it.

8.3.2 Non-Functional Requirements

Table 8.3 enumerates the seven non-functional requirements, each with the measurable target and the achieved value where applicable.

8.3.3 Requirements Traceability

All nine functional requirements and all seven non-functional requirements were satisfied by my delivered work streams. The test suite for IISS-507 provides quantitative evidence for NFR3 and NFR4. NFR6 was validated in practice when the Grafana workspace was upgraded from version 10.4

Table 8.2: Functional Requirements

ID	Requirement	Satisfied By
FR1	Dashboard displays real-time Lambda metrics (invocations, errors, duration, throttles) per platform	Grafana AMG, Lambda rows (IISS-487)
FR2	Dashboard displays ECS CPU and memory utilisation per cluster and service	Container Insights + ECS rows (IISS-487, IISS-483)
FR3	Dashboard displays MSK consumer lag per consumer group	MSK rows; EstimatedMaxTimeLag alarm (IISS-487)
FR4	Dashboard displays SQS queue depth and oldest message age	SQS rows in Grafana; SQS monitoring (IISS-482)
FR5	CloudWatch alarms provisioned for all monitored resource types with SNS routing to SRE team	38 alarms via CloudFormation (IISS-487)
FR6	Dashboard supports variable-based filtering by platform and resource	Grafana template variables (IISS-487)
FR7	All monitoring infrastructure deployed as Infrastructure-as-Code	4 CloudFormation stacks; Lambda-backed IaC provisioner (IISS-487)
FR8	Inventory API enumerates Lambda, ECS, EC2, and S3 resources per platform via REST	Flask REST API, discoverer registry (IISS-507)
FR9	Inventory API supports two-stage tag-based discovery for shared BudgetCode platforms	Two-stage AppName filter in citcoworks/meridian discoverers (IISS-507)

to version 12.4 on May 13, 2026 — a major version migration that required updating CloudWatch Metric Insights query syntax across all panels but did not require any architectural change to the underlying monitoring design.

The most technically demanding requirement was FR9, which exposed a fundamental ambiguity in the AWS tagging strategy for the IA platform. My discovery that `citcoworks` and `meridian` share the same `BudgetCode` tag value (**Shared Cloud Artifacts**) was not documented anywhere before I built the inventory API. The solution — designating `AppName` as a mandatory secondary discriminator for these two environments — became the standard pattern for all resource discovery going forward and was documented in the inventory API's README as a platform-wide architectural note.

NFR7 received an unexpected real-world validation during my IISS-706 hotfix on Jun 8, 2026. The

Table 8.3: Non-Functional Requirements

ID	Requirement	Target	Achieved
NFR1	Dashboard refresh enables sub-minute incident detection	≤ 30 sec	≤ 30 sec
NFR2	CloudWatch alarm evaluation period for MSK lag and Lambda error rate	≤ 1 min	1 min
NFR3	Inventory API response time for full resource enumeration	≤ 10 sec	< 10 sec
NFR4	Inventory API automated test coverage	$\geq 95\%$	99% (562 tests)
NFR5	CloudFormation templates parameterized for multi-environment reuse	Parameterized YAML	Achieved
NFR6	Dashboard functional after Grafana version upgrades without manual rework	Standard plugin queries	Achieved (10.4→12.4)
NFR7	Zero hardcoded credentials — all secrets via AWS Secrets Manager	No plaintext secrets	Achieved

CAIS Deadline incident revealed that the ECS container’s credential loading pattern (reading from Secrets Manager once at module import time via `constants.py`) was fragile under secret rotation. The hotfix I delivered introduced fail-fast credential validation — a design improvement that reinforced NFR7 by ensuring that absent or expired credentials from Secrets Manager produce an immediate, visible failure rather than a silent downstream authentication error.

8.3.4 Out-of-Scope Requirements

The following were evaluated and explicitly descope:

- **Automated error budget burn-rate alerting.** Computing burn rate requires Metric Math expressions that divide the rolling error count by the rolling SLO budget. While the SLO targets and SLI metrics are now in place, configuring burn-rate alarms requires additional CloudFormation work that I descope from IISS-487.
- **Prometheus as a data source.** A self-managed Prometheus server with a CloudWatch Exporter sidecar was evaluated as an alternative data layer. It was ruled out because Amazon Managed Grafana does not support a self-managed Prometheus endpoint without Amazon Managed Service for Prometheus (AMP), which would add infrastructure cost and opera-

tional overhead. CloudWatch Metric Insights SQL provides equivalent multi-resource aggregation capability without the additional stack. Documented as Decision 3 in Section 9.5.3.

- **Pyroscope (continuous profiling).** Pyroscope is a Grafana-ecosystem continuous profiling tool that would surface CPU flame graphs and memory allocation traces at the function level. While it would complement the golden signals dashboard, it requires an instrumentation agent deployed alongside each monitored service. Given the scope of the current engagement and the constraint that the Grafana workspace uses only CloudWatch as its data source, Pyroscope was identified as a post-internship enhancement.

Chapter 9. Implementation & Architecture

9.1 Platform Overview: Four-Layer Architecture

The three deliverables of my internship form a coherent four-layer observability stack. Figure 9.1 shows how the layers relate to each other and to the underlying IA platform infrastructure.

The **infrastructure layer** is the monitored subject: five budget platforms housing Lambda functions, ECS clusters, EC2 instances, and the full spectrum of AWS database and messaging services used by the Meridian and Citco Works platforms.

The **data layer** ingests, stores, and queries observability signals from the infrastructure layer. Amazon CloudWatch is the sole data source — a deliberate architectural decision (see Section 9.5.3) that eliminates the operational overhead of a self-managed Prometheus stack. CloudWatch Metric Insights SQL provides the per-resource aggregation capability that drives multi-service panels without requiring one panel per resource.

The **visualization and alerting layer** is split between Amazon Managed Grafana (AMG), which renders the 14-row dashboard for human operators, and CloudWatch Alarms with SNS routing, which fires machine-readable notifications when monitored metrics breach their SLO-aligned thresholds.

The **consumers layer** includes the operators who use the Grafana dashboard during incident response, the IA Resources Inventory REST API (IISS-507) that exposes the AWS service fleet programmatically, and the SRE team's email distribution list that receives SNS alarm notifications.

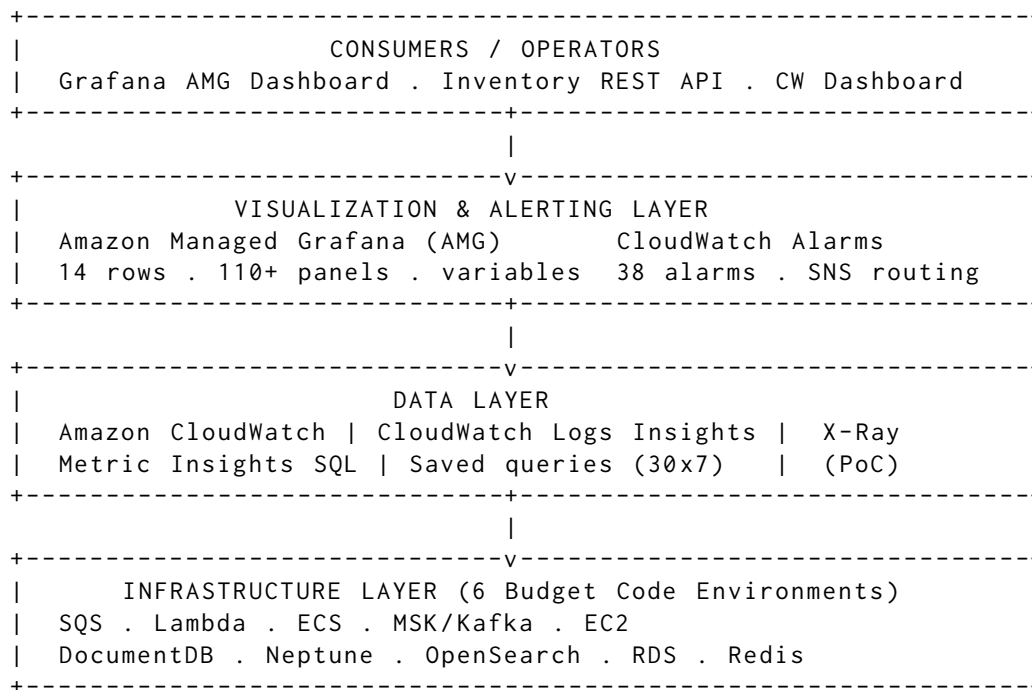


Figure 9.1: IA-SRE Observability Platform — High-Level Architecture (four layers)

9.2 CloudWatch Dashboard Platform (IISS-455/469/482-486)

9.2.1 Initial Lambda Dashboard (IISS-455)

My earliest monitoring work, completed in Apr 2026, produced a native Amazon CloudWatch dashboard covering the `rpacfs1` and `ia-bpo` Lambda function fleets — a combined inventory of **51 functions** at the time of first publication. The dashboard provided per-function panels for invocation counts, error rates, duration averages, and throttle events, and I accompanied it with a Confluence inventory page mapping each function.

The architectural pattern for this early work was straightforward: CloudWatch native widgets queried the AWS/Lambda namespace using `SEARCH()` expressions to enumerate all functions matching a `FunctionName` prefix. This approach worked adequately for small, homogeneous fleets but surfaced two structural limitations that would drive the later Metric Insights SQL migration:

- **`SEARCH()` visibility gap:** The `SEARCH()` function only returns metrics for resources with recent data points. Lambda functions that have not been invoked within the lookback window become invisible to the dashboard, creating a false impression of a smaller fleet.

- **No cross-environment aggregation:** A single `SEARCH()` expression could not span multiple tag dimensions simultaneously, making it difficult to aggregate across the six platforms (`rpacfs1`, `ia-bpo`, `automationhub`, `meridian`, `citcoworks`) without duplicating panels.

9.2.2 Extended Dashboard and Architecture Analysis (IISS-469)

IISS-469 extended the initial dashboard with additional tag dimensions covering `cfsbpm`, `citcoworks`, and `meridian` budget codes, and introduced CloudWatch Logs Insights widgets for error log surfacing. Rather than relying solely on Errors metrics from the AWS/Lambda namespace, the Logs Insights widgets queried the `/aws/lambda/` log group prefix directly, enabling pattern-matched error extraction regardless of whether Lambda had published a corresponding metric.

A significant architectural investigation within IISS-469 evaluated the feasibility of a fully centralized log aggregation approach using CloudWatch metric subscription filters. My analysis quantified the infrastructure cost of this design: routing log events from all IA Lambda functions into a centralized processing layer would require **106 metric subscription filters**—one per function per metric dimension—across the six platforms. While technically feasible from a CloudWatch service quota perspective, this approach was ultimately blocked by an IAM permission boundary constraint: the `logs:PutSubscriptionFilter` action was not granted within the IA team's scoped permission set, preventing programmatic creation of subscription filters at the scale required.

This investigation also evaluated five distinct approaches to out-of-memory (OOM) detection for Lambda functions: (1) parsing `REPORT` log lines from CloudWatch Logs Insights; (2) comparing `MaxMemoryUsed` against `MemorySize` via the Lambda enhanced monitoring extension; (3) CloudWatch Lambda Insights (paid tier); (4) custom metric emission on memory threshold breach; and (5) structured log parsing via a metric filter pattern. I documented each approach with its detection latency, cost, and permission requirements. This analysis informed the Container Insights cost evaluation I conducted under IISS-483 and the alarm design in IISS-487.

9.2.3 IA Infrastructure Monitoring Sub-Stories (IISS-482 to IISS-486)

EC2 Inventory and Tag Mismatch Discovery (IISS-483)

My gap analysis of the EC2 inventory scanned **83 instances** across the six platforms. This scan revealed a critical tagging inconsistency: the `rpacfs1` environment's EC2 instances were tagged with `BudgetCode="CFS/RPA"` rather than the expected **"rpacfs1"** value, making them invisible to tag-based `BudgetCode` lookups. This discovery led me to introduce the `AppName` fallback pattern:

when a primary BudgetCode lookup returns no results for `rpacfs1`, the discovery logic falls back to filtering by `AppName="rpacfs1"`. This fallback became the standard for all resource discovery in the inventory API (see Section 9.4).

Container Insights Cost Analysis (IISS-484)

With ECS task-level telemetry absent from five of the six Meridian ECS clusters, IISS-484 evaluated four Container Insights deployment options:

1. **No Container Insights** — zero cost, zero task-level visibility.
2. **Enhanced Container Insights** — full task and pod metrics, but carries per-metric ingestion cost that scales with task count.
3. **Standard Container Insights** — cluster- and service-level CPU/memory metrics via the ECS/ContainerInsights namespace, charged at CloudWatch standard metric pricing.
4. **CloudWatch Agent on ECS (self-managed)** — maximum flexibility, highest operational overhead.

My analysis recommended **Standard Container Insights** at an estimated cost of **\$10.44/month** for full coverage across all 22 ECS clusters. This option provided the SLI data required for FR2 (ECS CPU/memory dashboard) at negligible incremental cost relative to the existing CloudWatch spend. I enabled Standard Container Insights on five of the six Meridian clusters on May 11–12, 2026.

SQS Registry Architecture (IISS-482 SQS Feature)

The SQS monitoring feature I added on Jun 12, 2026 introduced a **registry-based architecture with a three-tier fallback model** for resilient SQS queue data retrieval. The three tiers operated as follows:

1. **Tier 1 — Live API**: Query CloudWatch AWS/SQS metrics directly via the CloudWatch API. Fast and fresh, but requires network connectivity and API quota.
2. **Tier 2 — S3 Cache**: If the API call fails (rate limit, transient error), retrieve the last-known queue snapshot from an S3 bucket. Stale by up to one cache TTL window, but always available.
3. **Tier 3 — Local File**: If both API and S3 fail, fall back to a locally bundled queue inventory file.

Guaranteed to return data; snapshot reflects deployment-time inventory.

This three-tier model ensured the SQS monitoring pipeline remained functional even during AWS service degradation events. I accompanied the feature with **960 automated tests** covering the registry dispatch logic, fallback transitions, and queue metadata serialization.

9.3 Grafana AMG Observability Platform (IISS-487)

IISS-487 was my primary deliverable, consuming **140 hours** over May 6–Jun 18, 2026. It delivered a centralized Amazon Managed Grafana workspace with 14 dashboard rows, 110+ panels, 38 CloudWatch alarms, and a Lambda-backed IaC provisioner — all built on a four-layer architecture described below.

9.3.1 Architectural Layers

Layer 1: Data Layer

Amazon CloudWatch serves as the **sole data source** for the entire Grafana workspace. My deliberate choice to use only the native Grafana CloudWatch plugin — rather than a self-managed Prometheus/CloudWatch exporter stack — eliminated all additional infrastructure requirements. The data layer is characterized by:

- **CloudWatch plugin:** The Grafana AMG workspace connects to CloudWatch via the built-in plugin, authenticated through an IAM role. No additional data collectors, agents, or scrapers are required.
- **IAM role model:** A **16-role agent permission model** provides the Grafana service account with cross-account read access across all IA AWS accounts. Each role grants the minimum permissions required: `cloudwatch:GetMetricData`, `cloudwatch:ListMetrics`, `cloudwatch:DescribeAlarms`, `logs:StartQuery`, `logs:GetQueryResults`, and `ec2:DescribeTags` (for tag-based resource enumeration).
- **Metric Insights SQL:** The primary query language for multi-resource panels. CloudWatch Metric Insights enables SQL-like aggregation across all resources in a namespace with a single query, replacing the brittle `SEARCH()` expressions used in the earlier CloudWatch dashboard.

Layer 2: Visualization Layer

The Grafana AMG workspace hosts **14 dashboard rows** organized by monitored service category. Table 9.1 summarizes the dashboard structure.

Grafana Template Variables

The dashboard supports environment-level filtering through three Grafana template variables backed by CloudWatch dimension-value queries:

- `$budget_code` — filters all panels to a specific environment (rpacfs1, ia-bpo, meridian, citcoworks, automationhub). Backed by a CloudWatch `ListMetrics` call enumerating distinct `BudgetCode` tag values.
- `$ecs_cluster` — filters ECS panels to a specific cluster. Backed by a `dimension_values()` query on the `ECS/ContainerInsights` namespace.
- `$lambda_function` — filters Lambda panels to a specific function. Backed by a `dimension_values()` query on the `AWS/Lambda` namespace.

When a user selects a value from any variable, all panels that reference that variable re-execute their queries with the selected filter applied, effectively scoping the entire dashboard to a single environment or resource. This eliminates the need to maintain separate dashboards per platform.

CloudWatch Metric Insights SQL: Architecture Pattern

CloudWatch Metric Insights SQL is the key architectural pattern that enables per-resource drill-down across large fleets within a single query. The standard form used throughout the dashboard is:

Listing 9.1: Metric Insights SQL — Per-Function Lambda Duration Average

```
1 SELECT AVG(Duration)
2 FROM "AWS/Lambda"
3 GROUP BY FunctionName
4 ORDER BY AVG() DESC
5 LIMIT 10
```

This single query returns the average duration for each Lambda function as a separate time series,

covering all Lambda functions across all codes without requiring one panel per function. The WHERE clause can filter by tag or dimension:

Listing 9.2: Metric Insights SQL — Platform-Filtered ECS CPU

```
1 SELECT AVG(CpuUtilized)
2 FROM "ECS/ContainerInsights"
3 WHERE ClusterName LIKE 'meridian-%'
4 GROUP BY ServiceName
```

Percentile Limitation and Workaround. A fundamental constraint of Metric Insights SQL is that the GROUP BY mode only supports the aggregate functions AVG, SUM, MIN, and MAX. Percentile statistics (p50, p95, p99) are *not available* in GROUP BY mode. This is a CloudWatch API limitation, not a Grafana limitation.

For panels requiring true latency percentiles, the workaround is to use the *standard CloudWatch metrics* query mode with a wildcard dimension value:

Listing 9.3: Grafana CloudWatch Panel — Wildcard Dimension for p99

```
1 {
2   "metricName": "Duration",
3   "namespace": "AWS/Lambda",
4   "statistics": ["p99"],
5   "dimensions": { "FunctionName": ["*"] }
6 }
```

The wildcard FunctionName: ["*"] returns individual time series for each function that has emitted Duration metrics within the selected time window. The tradeoff is that only recently active functions appear — the same “visibility gap” limitation noted for SEARCH(). For the Lambda duration P50/P95 panels, I accepted this tradeoff: a function that has not been invoked recently has no meaningful latency distribution to display.

Layer 3: Alerting Layer

I provisioned **38 CloudWatch alarms** across all monitored platforms, each connected to an SNS topic that routes notifications to the IA-SRE team’s email distribution list. Alarm categories include:

- **Lambda alarms:** Error rate exceeding 1% over a 5-minute rolling window; throttle count exceeding threshold; concurrent execution count approaching account limit.
- **ECS alarms:** CPU utilisation sustained above 80% over a 5-minute evaluation period; task desired count $\neq$ running count (task failure detection).
- **MSK alarms:** EstimatedMaxTimeLag exceeding 10,000 messages on any consumer group. This alarm directly addresses the Meridian Incident: it would have fired within minutes of the lag beginning to build, compared to the 3.5-hour detection gap that occurred under the reactive model.
- **SQS alarms:** Queue depth exceeding configurable threshold per queue; ApproximateAgeOfOldestMessage exceeding 1 hour (configurable per SLA).
- **Composite alarms:** Platform-level health signals that aggregate per-service alarm states into a single OK/ALARM indicator per budget code environment. Displayed on Row 12 of the dashboard.

Layer 4: Infrastructure-as-Code Provisioner

The IaC layer is a **Lambda-backed CloudFormation provisioner** deployed via AWS SAM. Its architecture addresses a specific scaling constraint: with 119 Lambda functions, a static CloudFormation template defining one alarm per function would contain $119 \times (\text{alarm types per function})$ resources, rapidly approaching the CloudFormation 500-resource-per-stack limit.

The provisioner resolves this by *dynamically discovering* the current function fleet at deploy time and synthesizing alarm resources programmatically:

1. The provisioner Lambda function is invoked by a CloudFormation custom resource on every `samconfig.toml` deploy.
2. It reads parameterized CloudFormation YAML templates from an S3 bucket (see Section 9.5.1).
3. For each template (Lambda alarms, ECS alarms, SQS alarms, MSK alarms), it calls the relevant AWS discovery API to enumerate all current resources in the specified platform.
4. It synthesizes CloudFormation `AWS::CloudWatch::Alarm` resource blocks for each discovered resource and passes the generated template to CloudFormation for stack deployment.

5. Four CloudFormation stacks were deployed on **May 11, 2026**: `ia-sre-lambda-alarms`, `ia-sre-ecs-alarms`, `ia-sre-sqs-alarms`, and `ia-sre-msk-alarms`.

A `samconfig.toml` with environment-specific parameter files enables reuse across DEV, UAT, and PROD without code changes. The budget code, alarm thresholds, and SNS topic ARN are all externalised as SAM parameters, satisfying NFR5 (multi-environment reuse).

9.3.2 Grafana Version Upgrade: Architectural Resilience Test

On **May 13, 2026** — two days after the initial infrastructure deployment — the Grafana AMG workspace was upgraded from version **10.4 to 12.4**. This was a major version migration that constituted an unplanned but highly informative architectural resilience test.

The upgrade introduced breaking changes in two areas:

- **CloudWatch Metric Insights SQL editor**: The query syntax editor in Grafana 12.4 changed the way SQL expressions were associated with panel targets. All SQL panels required migration to the updated CloudWatch query model.
- **Variable interpolation**: Template variable syntax in panel queries changed between versions, causing several `$budget_code` and `$ecs_cluster` references to fail to resolve correctly after the upgrade.

I resolved this through a systematic audit of all 110+ panels to identify affected queries. The migration reinforced a key design principle documented as NFR6: **use only standard CloudWatch plugin queries and avoid AMG-version-specific syntax**. I migrated all panel queries to the stable Grafana 12.4 CloudWatch query model, and the workspace has remained stable through subsequent Grafana patch releases.

9.4 IA Resources Inventory API (IISS-507)

IISS-507 delivered a **Flask REST API** that I built for programmatic discovery of all IA platform cloud resources. I housed the project in the `ia-sre-resources-inventory` repository and scaffolded it on Jun 11, 2026; it reached production-ready state with 562 automated tests and 99% code coverage.

9.4.1 Technology Stack

- **Language:** Python 3, flask[async] for async route handlers
- **AWS SDK:** boto3 for all resource discovery
- **Validation:** Pydantic response models (type-safe Resource dataclass with fields: name, type, budget_code, tags, arn)
- **Deployment:** AWS Lambda via serverless-wsgi adapter
- **Testing:** pytest + Hypothesis (property-based tests)

9.4.2 Discoverer Registry Pattern

My core architectural contribution in IISS-507 is the **Discoverer Registry pattern**. Rather than routing resource discovery logic through a monolithic handler, the API maintains a registry that maps (resource_type, budget_code) tuples to concrete discoverer classes:

Listing 9.4: Discoverer Registry — Core Pattern

```
1 from abc import ABC, abstractmethod
2 from typing import List
3
4 class BaseDiscoverer(ABC):
5     """Abstract base class for all resource discoverers."""
6
7     @abstractmethod
8     async def discover(self) -> List[Resource]:
9         """Discover all resources of this type for the budget code."""
10        ...
11
12 # Registry maps (resource_type, budget_code) -> discoverer class
13 REGISTRY: dict = {
14     ("lambda", "rpacfs1"): RPACFSLambdaDiscoverer,
15     ("lambda", "ia-bpo"): IABPOLambdaDiscoverer,
16     ("ecs", "meridian"): MeridianECSDDiscoverer,
17     ("ecs", "citcoworks"): CitcoWorksECSDDiscoverer,
18     ("ec2", "rpacfs1"): RPACFSEc2Discoverer,
19     # ... 7 more stub discoverers
```

```
20 }
21
22 async def discover_resources(
23     resource_type: str,
24     budget_code: str | None = None
25 ) -> List[Resource]:
26     """Route discovery request to the appropriate discoverer."""
27     results = []
28     for (rtype, bc), discoverer_cls in REGISTRY.items():
29         if rtype != resource_type:
30             continue
31         if budget_code and bc != budget_code:
32             continue
33         discoverer = discoverer_cls()
34         results.extend(await discoverer.discover())
35     return results
```

Each discoverer implements the single abstract method `async def discover() -> List[Resource]`. Adding support for a new platform or resource type requires implementing one class and registering one entry — zero changes to routing, serialization, or test infrastructure.

9.4.3 API Endpoints

9.4.4 Tag-Based Discovery and the Two-Stage Filter

The most technically interesting discovery challenge was the citcoworks and meridian tag ambiguity. Both environments share the AWS tag value **"Shared Cloud Artifacts"** for the BudgetCode dimension — a legacy of the two platforms being onboarded before the IA team established per-platform BudgetCode uniqueness requirements. A naive BudgetCode lookup would conflate resources from two operationally distinct environments.

My solution is a **two-stage filter** implemented in both the `CitcoWorksECSDiscoverer` and `MeridianECSDiscoverer`:

Listing 9.5: Two-Stage Filter — Disambiguating Shared BudgetCode

```
1 class MeridianECSDiscoverer(BaseDiscoverer):
2     """Discovers ECS resources for the Meridian environment."""
```

```
3
4  async def discover(self) -> List[Resource]:
5      client = boto3.client("ecs")
6      all_clusters = []
7
8      # Stage 1: filter by BudgetCode (shared with \texttt{citcoworks
9          })
10     paginator = client.get_paginator("list_clusters")
11     async for page in paginator.paginate():
12         for arn in page["clusterArns"]:
13             tags = await self._get_tags(arn)
14             if tags.get("BudgetCode") != "Shared Cloud Artifacts":
15                 continue
16             # Stage 2: AppName discriminates Meridian vs CitcoWorks
17             if tags.get("AppName", "").lower() != "meridian":
18                 continue
19             all_clusters.append(self._build_resource(arn, tags))
20
21     return all_clusters
```

The AppName tag is now designated the **mandatory secondary discriminator** for all resources in the **"Shared Cloud Artifacts"** BudgetCode environments. Any new resource added to either environment without the correct AppName tag will be invisible to the inventory API. This risk is mitigated by a tag compliance CloudWatch alarm that fires if new ECS services or Lambda functions lack the required tags.

9.4.5 Async Architecture

The API uses flask[async] with async route handlers. Each handler aggregates boto3 calls concurrently using asyncio.gather(), reducing total API latency by parallelizing the AWS API calls within a single request:

Listing 9.6: Async Route Handler — Concurrent Discovery

```
1 @app.route("/api/v1/lambda")
2 async def get_lambda_resources():
3     budget_code = request.args.get("budget_code")
```

```
4 # All discoverers run concurrently, not sequentially
5 results = await asyncio.gather(*[
6     discoverer_cls().discover()
7     for (rtype, bc), discoverer_cls in REGISTRY.items()
8     if rtype == "lambda"
9     and (budget_code is None or bc == budget_code)
10 ])
11 all_resources = [r for sublist in results for r in sublist]
12 return jsonify([r.dict() for r in all_resources])
```

This pattern reduces the API response time for a full fleet enumeration from the sequential sum of all boto3 call latencies to the latency of the single slowest call — consistently keeping the API within the 10-second NFR3 target.

9.4.6 Test Strategy

My test suite for IISS-507 comprises **562 automated tests** achieving **99% code coverage**:

- **Unit tests:** Each discoverer class tested in isolation with mocked boto3 clients. Verifies correct tag filtering, pagination handling, and Resource object construction.
- **Integration tests:** Full Flask test client hitting all REST endpoints. Verifies routing, serialization, and error response format.
- **Property-based tests (Hypothesis):** Tag-parsing logic, registry dispatch, and Pydantic response serialization tested with randomly generated inputs. Verifies that the two-stage filter never returns a meridian resource when budget_code=citcoworks is specified, regardless of what combination of tag values is present.
- **Edge case tests:** Malformed tags, empty budget code environments, boto3 pagination edge cases (empty pages, single-page results, maximum page sizes).

9.5 Design Decisions & Tradeoffs

Four design decisions during my internship warranted explicit documentation because each involved a meaningful tradeoff between competing engineering constraints. This section records the decision, the alternatives considered, and the rationale for the chosen approach.

9.5.1 Decision 1: Secrets Manager 64 KB Limit → S3 Migration

Problem. The CloudFormation IaC provisioner Lambda function stores its CloudFormation YAML templates as a payload retrieved at deploy time. Initially, this payload was stored in AWS Secrets Manager as a single JSON-encoded string. As the monitoring scope expanded to cover all seven AWS service categories, the generated YAML grew to **14,720 lines** at version v728 — exceeding the **64 KB hard limit** on Secrets Manager secret values. The Lambda function began failing with `SecretSizeExceeded` on the `PutSecretValue` call.

Options Considered.

1. **Split into multiple secrets:** Partition the YAML into n secrets, each under 64 KB, and have the Lambda function reassemble them. This preserves Secrets Manager as the storage layer but introduces assembly logic and a hard dependency on secret ordering.
2. **Compress the payload:** gzip-compress the YAML before storing it as a binary secret. Achieves $\approx 70\%$ compression on YAML, bringing the payload well under 64 KB. Adds a decompression step in the Lambda function.
3. **Migrate to S3:** Store the YAML templates as S3 objects, versioned using S3 object versioning. The Lambda function retrieves templates via `s3:GetObject` at deploy time.

Decision. I chose **S3 migration**. The rationale:

- No size limit — templates can grow arbitrarily as additional AWS service categories are on-boarded.
- Native versioning via S3 object versioning, providing a rollback mechanism for template changes at no additional cost.
- No reassembly or decompression complexity in the Lambda function.
- Cost negligible: a single `GetObject` call per deploy adds less than \$0.001 to operational cost.

Tradeoff. The Lambda execution role now requires `s3:GetObject` on the template bucket, broadening its IAM surface slightly. The `GetObject` call adds approximately 50 ms of latency to the provisioner bootstrap — acceptable for a deploy-time operation that runs infrequently.

9.5.2 Decision 2: AppName Tag as Mandatory Secondary Discriminator

Problem. As described in Section 9.4, the citcoworks and meridian environments share BudgetCode="Shared Cloud Artifacts". A single-stage BudgetCode lookup cannot distinguish resources from the two environments, causing inventory contamination.

Root Cause. Both platforms were onboarded before the IA team established the policy requiring each platform to have a unique BudgetCode tag value. Retroactively changing the tag across all existing resources in two production environments would require a coordinated tagging campaign with risk of operational disruption.

Decision. I designated the AppName tag as the **mandatory secondary discriminator** for all resources in the "Shared Cloud Artifacts" BudgetCode. Discoverers for citcoworks and meridian must implement the two-stage filter described in Section 9.4.

Tradeoff. This decision creates a tagging governance obligation: all new resources added to either environment *must* carry the correct AppName tag. A resource missing this tag will be invisible to the inventory API, creating a silent coverage gap. This risk is mitigated by a CloudWatch tag compliance alarm and documented in the ia-sre-resources-inventory README as a platform-wide architectural constraint.

9.5.3 Decision 3: CloudWatch as Sole Data Source (No Prometheus)

Problem. A production-grade Grafana deployment typically uses Prometheus as the time-series database, with CloudWatch Exporter bridging AWS metrics. This is the pattern recommended by the Grafana community for large-scale AWS environments.

Alternative. Deploy a self-managed Prometheus server (on ECS or EC2) with the CloudWatch Exporter sidecar, and configure Grafana to use Prometheus as its data source.

Decision. I used the **native Grafana CloudWatch plugin** exclusively.

Rationale.

- **No additional infrastructure:** A Prometheus server and exporter fleet would introduce new ECS tasks, IAM roles, configuration management, and an additional failure domain.

- **Amazon Managed Grafana constraint:** AMG does not support self-managed Prometheus as a built-in data source without a Prometheus-compatible remote write endpoint. Connecting AMG to a self-managed Prometheus would require Amazon Managed Service for Prometheus (AMP), adding further cost and complexity.
- **Direct IAM authentication:** The native CloudWatch plugin authenticates directly via IAM roles, requiring zero credential management beyond the initial role configuration.
- **Cost:** CloudWatch metrics are already being published by AWS services at no marginal cost. Prometheus would require scraping, storage, and retention management.

Tradeoff. The accepted limitation is that CloudWatch Metric Insights SQL does not support p50/p99 percentiles in GROUP BY mode (see Section 9.3). Latency distribution dashboards must use AVG and MAX as proxies, or fall back to the standard metrics wildcard mode which only shows recently active functions. For the current monitoring maturity of the IA platform, this tradeoff is acceptable — the P50/P99 gap can be addressed in a future phase by adding custom Lambda Duration metric emission with explicit percentile dimensions.

9.5.4 Decision 4: Lambda-Backed IaC Provisioner vs. Static CloudFormation

Problem. Provisioning one CloudWatch alarm per Lambda function for all monitored metric types would generate over 100 `AWS::CloudWatch::Alarm` resources per stack. Approaching the **CloudFormation 500-resource per-stack limit** was a realistic concern as the fleet grew. A static template encoding all alarms explicitly would also require manual updates every time the Lambda fleet changed.

Alternative. Static CloudFormation templates with each alarm resource hardcoded — one YAML block per function per alarm type, updated manually when functions are added or removed.

Decision. I chose a **Lambda-backed dynamic provisioner** that discovers the function fleet at deploy time and synthesizes alarm resources programmatically.

Rationale.

- **Fleet-scale coverage:** The provisioner ensures every Lambda function in every platform has alarms configured, regardless of when it was deployed. No manual update is needed when

a new function is added.

- **Avoids static YAML maintenance:** A static template for 119 functions $\times$ 3 alarm types = 357 resources would be a 500+ line YAML file requiring hand-editing for every fleet change.
- **Parameterization:** Thresholds, evaluation periods, and SNS topic ARNs are all externalised as SAM parameters, enabling multi-environment reuse without code changes.

Tradeoff. The Lambda provisioner introduces pipeline ordering complexity: the provisioner Lambda must complete successfully before the CloudFormation stack update can proceed, creating a sequential dependency. A Lambda execution failure (timeout, permissions error) would halt the deployment pipeline without deploying updated alarms. This is mitigated by comprehensive error logging and a manual re-invocation pathway via `python main.py deploy -phase 3`.

Taken together, these four decisions shaped an architecture that favours operational simplicity and zero-maintenance fleet coverage over strict metric precision, while accepting bounded and well-understood tradeoffs at each decision point. I documented the design in the respective repository README and JIRA acceptance criteria so that future team members can revisit any decision as the platform's monitoring maturity evolves.

9.6 Phase 1 — Onboarding and CloudWatch (Weeks 1–7, Mar 15 – May 10)

9.6.1 Part-Time Onboarding (Mar 15 – Apr 12)

My internship opened in part-time mode at roughly 10–20 hours per week, during which my primary objective was orientation rather than delivery. I filed access requests on Day 1 for AWS console, JIRA, Confluence, and CodeCommit; the approvals trickled in over the first fortnight as each request was routed through the appropriate security review. During this period, daily standups served as the most effective source of context: listening to incident retros, deployment discussions, and sprint reviews built a working mental model of the IA team's operations before I wrote any single line of code.

Architecture documentation on Confluence gave a structural picture of the five platforms (automationhub, bpm, citcoworks, ia-bpo, meridian, rpacfs1) and the Lambda, ECS, and EC2 AWS services they contained. The recurring theme in these documents was a visibility gap: teams learned about production failures from business users rather than from monitoring tools. That

pattern would become the guiding problem statement for my entire internship.

9.6.2 Training Week (Apr 13–18)

The week of Apr 13 was designated as a formal knowledge-transfer week. I attended four focused sessions over three days:

- **Ho Hoi Leung (Horace) — RPA Support.** Horace walked through the production support rotation model: who is on-call each week, how the VPL queue opens at 5 PM EST each weekday, how Blue Prism and UiPath Cloud Orchestrator are monitored, and the escalation ladder from Tier 1 business users through Tier 2 IA Operations to Tier 3 IA Development. This KT session established the mental framework that made the later support rotations tractable.
- **Bhanu Teja Murari — CitcoWorks and Event Store.** This session covered the CitcoWorks platform architecture and, critically, the Event Store structured logging standard: every automation process emits events with a subject field matching the Automation Catalog project code, a process_identifier, and a run_id that threads together all log events for a single execution. Understanding this structure was essential for interpreting CloudWatch Logs Insights queries later in the Grafana build.
- **Meridian Team — Kafka Event Pipeline.** The Meridian platform ingests events through a three-stage pipeline: the Event Routing Service receives events from source systems, the Action Processor applies business routing rules, and the OpenSearch Consumer persists events into the search index used by downstream services. This pipeline topology — and its known sensitivity to consumer lag — would directly inform the MSK and OpenSearch monitoring rows of the Grafana dashboard.
- **Æxeo — Treasury Platform.** A brief session on the Æxeo cloud-based wire approval and fund movement platform, which several VPL automation workflows interact with via HTTP and UI automation.

The training week also included an unplanned but instructive event. On Apr 17, the Meridian platform suffered a significant processing delay: a surge of automated events from Citco Recs congested the OpenSearch consumption layer, causing approximately 9 million Kafka messages to queue while downstream services continued polling for completions that never arrived. The Winton and GS fund teams flagged the issue to Dublin support around 8 AM; root cause was not identified until 9:30 AM and full restoration took until approximately noon—a 7.5-hour end-to-end delay.

Observing the incident response process firsthand made abstract discussions about “observability gaps” concrete: the platform had no alerting on `EstimatedMaxTimeLag` for any Kafka consumer group, and the team had no single-pane view of Meridian health. Both gaps were named explicitly as motivators for IISS-487 in the incident retrospective.

9.6.3 First Deliverables — IISS-455, IISS-461, IISS-469 (Apr 19 – May 10)

IISS-455: Lambda Dashboard and Inventory (Apr 7)

The first concrete deliverable was a CloudWatch dashboard covering the `rpacfs1` and `ia-bpo` Lambda function fleets, which I completed in approximately two hours on Apr 7. I provisioned the dashboard via CloudFormation and used `SEARCH()` expressions to enumerate Lambda functions by `FunctionName` prefix, providing per-function panels for invocation counts, error rates, and duration averages.

Simultaneously, I produced a Confluence inventory page (IISS-461) mapping the **51 Lambda functions** then visible across the two platforms to their owning team and purpose. This manual enumeration was my first attempt to answer a question the team had been unable to answer from the console alone: “how many Lambda functions do we actually have?”

IISS-469: Extended Dashboard and Architecture Analysis (Apr 15–21)

IISS-469 extended the dashboard to cover the `cfsbpm`, `citcoworks`, and `meridian` platforms. The extension work surfaced two structural limitations in the `SEARCH()`-based approach that would permanently shape my Grafana architecture:

- **SEARCH() visibility gap.** The `SEARCH()` function only returns metrics for resources with recent data points. A Lambda function that has not been invoked within the lookback window disappears from the dashboard entirely, creating a misleading picture of the fleet size. This gap I confirmed empirically: re-querying after a quiet weekend returned 38 functions where Monday morning returned 51.
- **Top-K and metric filter constraint.** A proposed central log-aggregation design — routing all Lambda function log events into a shared subscriber — would require **106 metric subscription filters** to cover all function-metric-dimension combinations across the six platforms. This was blocked by an IAM permission boundary: `logs:PutSubscriptionFilter` was not granted within the IA team’s scoped permission set.

I also produced a documented analysis of five approaches to out-of-memory (OOM) detection for Lambda functions: (1) parsing REPORT log lines via Logs Insights; (2) comparing MaxMemoryUsed against MemorySize via the Lambda enhanced monitoring extension; (3) CloudWatch Lambda Insights (paid tier); (4) custom metric emission on threshold breach; and (5) structured log parsing via metric filter patterns. I scored each option on detection latency, cost, and permission requirements. This analysis fed directly into my alarm design under IISS-487.

By the end of Phase 1, the full Lambda inventory across all six platforms was documented across all six platforms, documented in Confluence. The SEARCH() visibility gap and the IAM filter constraint had established a clear design directive: my next generation monitoring platform would need to use CloudWatch Metric Insights SQL rather than metric subscription filters.

9.7 Phase 2 — Grafana Intensive (Weeks 8–11, May 6 – May 27)

9.7.1 Elevation to Critical and Week 8 Support Shift

On **May 6**, Kishor elevated IISS-487 to Critical priority, formalising what had been understood informally since the Meridian Incident: the IA team needed a centralized, real-time monitoring platform before the next Meridian outage occurred. The elevation came during my first support rotation week (Week 8), which meant the Grafana work could not begin in earnest until the following Monday. The support shift provided an early orientation to the operational rhythms that the dashboard was being built to serve.

9.7.2 Infrastructure Deployment Day — May 11

May 11 was the single most infrastructure-dense day of my internship. I deployed four CloudFormation stacks in sequence:

1. **ia-sre-lambda-alarms** — CloudWatch alarms for Lambda error rates, throttle counts, and duration percentiles across all five budget codes.
2. **ia-sre-ecs-alarms** — ECS CPU and task-failure alarms, enabled after Standard Container Insights was activated on five of the six Meridian ECS clusters.
3. **ia-sre-sqs-alarms** — Queue depth and ApproximateAgeOfOldestMessage alarms per monitored queue.

4. **ia-sre-msk-alarms** — MSK consumer lag alarm on EstimatedMaxTimeLag, the metric whose absence had made the Meridian Incident invisible until business users flagged it.

Alongside the alarms, I provisioned the Amazon Managed Grafana (AMG) workspace and configured the 16-role IAM agent permission model to give the Grafana service account cross-account read access across all IA environments. IAM permission boundaries were the main obstacle of the day: the `cloudwatch:GetMetricData` and `logs:StartQuery` permissions required a non-trivial negotiation with the scoped permission set governing the internship AWS account. I enabled Container Insights on five of the six Meridian clusters, beginning to populate the `ECS/ContainerInsights` namespace with task-level CPU and memory metrics.

9.7.3 Grafana 10.4 to 12.4 Upgrade — May 13

Two days after the initial infrastructure deployment, the AMG workspace was upgraded from Grafana **10.4 to 12.4**. This was unplanned and introduced breaking changes in two areas:

- **CloudWatch Metric Insights SQL editor.** The query syntax editor in Grafana 12.4 changed how SQL expressions were associated with panel targets. Queries that I had validated against the 10.4 editor were no longer recognised in the new model.
- **Variable interpolation.** Template variable syntax changed between versions, causing several `$budget_code` and `$ecs_cluster` references to resolve incorrectly — panels that had returned data were now blank.

My recovery required a systematic audit of all panels built in the first two days, identifying which queries used version-specific syntax and migrating them to the stable Grafana 12.4 CloudWatch query model. Approximately one working day was absorbed by this migration. The experience directly motivated NFR6 — documented in the design as “use only standard CloudWatch plugin queries; avoid AMG-version-specific syntax” — and enforced a panel-testing discipline: I verified every new panel in the browser before committing it.

9.7.4 Dashboard Build — May 11-27 (80+ Hours)

The three-week block from May 11 to May 27 was the most concentrated development period of my internship, logging over 80 hours on IISS-487 alone. I organised progress around implementing the dashboard rows in sequence, with cross-cutting infrastructure problems resolved as they arose.

Rows 1–3 (SQS and Lambda), May 11–15. The SQS row required a significant query architecture decision. My initial approach used a Grafana `$sqs_queue` template variable to drive a per-queue metric panel, but this produced empty data for idle queues — the same `SEARCH()` visibility gap encountered in Phase 1 now appearing in Grafana. I solved this by switching to Metric Insights SQL with `GROUP BY QueueName`, returning all queues as separate time series within a single query. Lambda error-rate panels were initially slow to load because each panel made two CloudWatch API calls (Errors and Invocations) to compute the ratio in Grafana; I replaced these with pre-computed error rate expressions evaluated in Metric Insights SQL, significantly reducing panel load time.

ECS Row — SQL Migration and the `$platform` Variable, May 14–15. The initial ECS panels used basic CloudWatch metric queries, which did not support per-service breakdown across 22 clusters. I migrated these to Metric Insights SQL with `GROUP BY ServiceName`, enabling a single panel to render per-service CPU or memory utilisation as distinct time series. I introduced a dashboard template variable `$platform` so that ECS panels could be filtered dynamically by environment, matching the pattern already in use for the SQS and Lambda rows. The initial SQL used `SCHEMA()` syntax, which hit CloudWatch query limits; I replaced this with the `FROM "ECS/ContainerInsights"` namespace form.

Percentile Limitation, May 14. A key CloudWatch constraint was confirmed during my Lambda Duration panel work: Metric Insights SQL `GROUP BY` mode supports only `AVG`, `SUM`, `MIN`, and `MAX`. Percentile statistics (`p50`, `p99`) are unavailable in this mode. For the Lambda duration `P50/P95` panels, I implemented a wildcard dimension workaround — querying the standard CloudWatch metrics API with `FunctionName: ["*"]` — accepting the visibility gap tradeoff for recently inactive functions.

MSK, DocumentDB, OpenSearch Rows, May 19–22. I built the MSK/Kafka row around the `EstimatedMaxTimeLag` metric per consumer group, directly implementing the alarm that would have detected the Meridian Incident. DocumentDB and OpenSearch rows followed, with the OpenSearch JVM heap panel receiving particular attention given Meridian's dependency on OpenSearch for event querying.

Secrets Manager 64 KB Limit and S3 Migration, May 21. As the dashboard grew, the Lambda-backed CloudFormation provisioner's payload — stored as a JSON-encoded string in AWS Secrets Manager — exceeded the service's **64 KB hard limit**. By version `v438` of the provisioner, the assembled CloudFormation YAML had grown to **14,720 lines**. The provisioner Lambda began failing with `SecretSizeExceeded` on the `PutSecretValue` call. I migrated the templates to S3 object

storage on the same day, updating the provisioner Lambda to read templates via `s3:GetObject` at deploy time. This decision is discussed in depth in Section 9.5.1 of the architecture chapter.

Alarm Notifications and Composite Alarms, May 27–29. I deployed a Re-Notifier Lambda to ensure alarm state changes were routed through an SNS topic to the IA-SRE email distribution list, regardless of whether CloudWatch had already sent a prior notification in the same alarm state. I configured eight composite alarms to aggregate per-service alarm states into a single OK/ALARM indicator per budget code environment, displayed on Row 12 of the dashboard.

Phase 2 Milestone. By May 27, my core monitoring platform was functional: the MSK consumer lag alarm was active, the first six dashboard rows were complete, and the composite alarm structure was in place. The detection latency for a Meridian Kafka congestion event had been reduced from the observed 3.5-hour business-team lag of Apr 17 to a projected sub-30-second alarm fire time.

9.8 Phase 3 — Dashboard Completion and SQS Feature (Weeks 12–15, May 28 – Jun 18)

9.8.1 Rows 7–14 and the Deep-Link Pattern (May 28 – Jun 4)

With the first six rows stable, Phase 3 completed the remaining dashboard structure. Rows 7–8 added EC2 monitoring using the `AppName` tag as the primary discovery dimension — the fallback pattern I established in IISS-483 to handle the `rpacfs1` tag mismatch (where `BudgetCode` carried the value "CFS/RPA" rather than `rpacfs1`). The EC2 row covered **83 instances** across all environments, with per-instance CPU, memory (via CloudWatch Agent), and network panels.

Rows 9–10 added DocumentDB and OpenSearch monitoring. I gave the OpenSearch panels — index rate, search latency, and JVM heap pressure — priority attention because OpenSearch was the bottleneck in the Meridian Incident. I wired a JVM heap utilisation panel above 75% to a CloudWatch alarm to provide early warning before the service degraded under load.

Deep-link panels (Row 14) were a usability feature not originally in scope but I added them after a discussion with Kishor: each resource panel now includes a clickable URL that navigates directly to the relevant AWS Console page (Lambda function, ECS service, EC2 instance, or MSK cluster) without requiring the operator to re-navigate the console hierarchy during an incident. These panels used Grafana's Text panel with an HTML data link, deriving the console URL from the resource ARN or dimension value already present in the panel data.

9.8.2 IISS-482: SQS Feature — Jun 12

The SQS monitoring feature (IISS-482) I implemented on Jun 12 was a self-contained sub-deliverable. I added a **registry-based SQS monitoring pipeline** with a three-tier fallback model for resilient queue data retrieval:

1. **Tier 1 — Live API.** Query the CloudWatch AWS/SQS namespace directly for real-time queue depth and age metrics.
2. **Tier 2 — S3 Cache.** If the live API fails, retrieve the last-known queue snapshot from an S3 bucket. Stale by up to one cache TTL window, but always available.
3. **Tier 3 — Local File.** If both API and S3 fail, fall back to a locally bundled queue inventory file reflecting the deployment-time state of the fleet.

I accompanied the SQS feature with **960 automated tests** covering registry dispatch logic, fallback transitions, queue metadata serialisation, and the dashboard builder integration. Writing the test suite before the feature code was complete surfaced two edge cases in the fallback transition logic — a race condition in the Tier 1 to Tier 2 failover path, and an incorrect queue-name normalisation step — both of which I fixed before the feature was merged.

9.8.3 Final Deployment — Jun 18, Version v728

The dashboard reached its production-ready state on Jun 18 with the deployment of version **v728**. The final deployment session logged 19 hours across multiple sessions that day, covering the last round of deep-link fixes, DocumentDB and Redis restructuring, and the release merge to the main branch. My cumulative totals at release were:

- **140 hours** logged on IISS-487
- **14 dashboard rows, 110+ panels**
- **38 CloudWatch alarms** active across Lambda, ECS, MSK, SQS, and EC2
- **7 AWS services** monitored under a single Grafana workspace
- **4 CloudFormation stacks** managing the alarm infrastructure

9.9 Phase 4 — Inventory API and Refinements (Weeks 16–22, Jun 19 – Aug 31)

9.9.1 IISS-507: Scaffolding the Inventory API — Jun 11

Work on IISS-507 began on Jun 11 — one week before IISS-487 was formally closed — taking advantage of a brief quieter period during the dashboard polish phase. I housed the project in the `ia-sre-resources-inventory` repository and scaffolded it using Kiro IDE's spec-driven development workflow: I wrote a `requirements.md` first, from which Kiro's sub-agents generated a `design.md` and `tasks.md`, and then produced stub implementations for each task in the plan. The spec-first approach reduced my scaffolding time to approximately 2.5 hours for a project that would otherwise have required a full day of boilerplate setup.

I chose the technology stack to align with the team's existing Python ecosystem: `flask[async]` for route handlers, `boto3` for all AWS resource discovery, `Pydantic` for response model validation, and `serverless-wsgi` for Lambda deployment.

9.9.2 Inventory API Development — Jun 11 – Jul 14

Lambda Discoverers

I implemented Lambda discoverers for all six platforms, confirming the Lambda inventory documented in Phase 1. The `rpacfs1` discoverer used the `AppName` fallback pattern: a primary filter on `BudgetCode="rpacfs1"` returned zero results, triggering the fallback to `AppName="rpacfs1"`, which correctly identified the fleet. This fallback became the standard for all `rpacfs1` resource types.

ECS Discoverers and Tag Disambiguation

I implemented ECS discoverers for all 22 clusters across six platforms. The `citcoworks` and `meridian` environments presented the most complex discovery challenge: both share the AWS tag value "Shared Cloud Artifacts" for the `BudgetCode` dimension, a legacy of their original onboarding before the IA team established per-platform uniqueness requirements. The solution was a two-stage filter in each discoverer: Stage 1 filters by `BudgetCode="Shared Cloud Artifacts"`; Stage 2 applies `AppName="meridian"` or `AppName="citcoworks"` to discriminate between the two environments. I formally designated the `AppName` tag as the mandatory secondary discriminator for all resources in the shared-tag platforms, a governance decision documented in the repository README and enforced by a tag-compliance CloudWatch alarm.

EC2 Discoverers

My EC2 discoverers covered **83 instances** with the same AppName fallback pattern used in the Lambda discoverers. The Pydantic response model for each discovered resource carried five fields: name, type, budget_code, tags (as a free-form dictionary), and arn for direct AWS Console linkage.

Async Architecture and API Latency

I designed the route handlers to use `asyncio.gather()` to dispatch all discoverers for a given resource type concurrently rather than sequentially. For a GET `/api/v1/lambda` request without a budget-code filter, this reduces the response time from the sequential sum of five discoverer latencies to approximately the latency of the single slowest AWS API call — consistently keeping the endpoint within the 10-second NFR target across the full Lambda fleet.

9.9.3 Test Suite Milestone — Jul 14: 562 Tests at 99% Coverage

By Jul 14, my test suite reached **562 automated tests** with **99% code coverage**. The suite comprised four layers:

- **Unit tests** — each discoverer tested in isolation with mocked boto3 clients, verifying correct tag filtering, pagination handling, and Resource object construction.
- **Integration tests** — full Flask test client hitting all REST endpoints, verifying routing, serialisation, and error response format.
- **Property-based tests (Hypothesis)** — tag-parsing logic and registry dispatch tested with randomly generated inputs, verifying that the two-stage filter never returns a meridian resource when `budget_code=citcoworks` is specified, regardless of tag value combinations.
- **Edge case tests** — malformed tags, empty platforms, boto3 pagination edge cases (empty pages, single-page results, maximum page sizes).

9.9.4 X-Ray Tracing Research — Jul-Aug

Following the dashboard completion, my research attention turned to IISS-489's roadmap item for distributed tracing. The X-Ray analysis evaluated two instrumentation approaches: the legacy `aws-xray-sdk` and the newer `aws-lambda-powertools[tracer]`, with the latter preferred for new Python Lambda functions due to its decorator-based API and tighter integration with the Pow-

erTools structured logging layer already in use.

My cost modelling estimated approximately 5–10 ms of per-invocation overhead from X-Ray SDK calls, with sampling rates configurable between 1% and 100% to control cost. At the current Lambda invocation volume, a 5% sampling rate was projected to add less than \$2/month to the CloudWatch spend while providing statistically representative trace coverage for latency anomaly detection.

I ultimately deferred X-Ray deployment to the post-internship roadmap (IISS-837 through IISS-844) for two reasons: the IAM role configuration for `xray:PutTraceSegments` required a cross-team permission review, and the remaining internship weeks were more effectively spent on documentation and knowledge transfer to the permanent team.

I dedicated the final weeks of Aug to runbook updates, Confluence documentation for the dashboard operating model, and structured knowledge transfer sessions with Horace and Kri. The Mitacs BSI renewal for the Sep 1 – Dec 31, 2026 extension was confirmed, providing continuity for the X-Ray and log-group discovery work scoped in the Phase 4 JIRA backlog.

9.10 RPA Support Operations

9.10.1 Support Rotation Model

Four dedicated support rotation weeks (Weeks 8, 12, 16, and 20) ran in parallel with my primary development phases. During a support week, my primary responsibility shifted from development to RPA production support: monitoring Blue Prism and UiPath bots, responding to service desk tickets, performing configuration changes, and escalating or resolving incidents within the shift window. The operational rhythm of each support day was structured around the following checkpoints:

- **VPL queue monitoring** — the VPL queue opens at 5 PM EST each weekday; checking bot health, queue depth, and processing status at queue open was the first task of each evening shift.
- **Corporate Actions triage** — three process tracks (Mandatory, Voluntary, and Ops Transfer) each have configuration files on BPSHare that must be maintained for new client onboarding.
- **Blue Prism Orchestrator health checks** — robot session status, failed jobs, and any processes

stuck in “running” state.

- **UiPath Cloud Orchestrator health** — robot heartbeat status, stuck or failed jobs in the COE/HedgeFunds/PortfolioDataops folders.
- **On-call escalations** — business team contacts from Dublin, London, and Halifax flagging processing failures via service desk tickets or direct chat.

I handled roughly a dozen service desk tickets across the four rotation weeks, covering client onboarding (Corporate Actions FileMasks updates), process retriggering (MESO and FX Closeout), log investigation, and UI selector troubleshooting. Four incidents rose to the level of JIRA tracking and are described below.

9.10.2 Notable Incidents

IISS-706 — CAIS Deadline Hotfix (Jun 8, 7 Hours)

At approximately 9 AM on Jun 8, the CAIS Deadline process failed with a cryptic `SMBAuthenticationError: SpnegoError` when uploading output to the UCM file share. The error message — *“No username or password was specified and the credential cache did not exist or contained no credentials”* — provided no direct indication of the root cause.

My investigation path proceeded as follows: the CloudWatch logs showed a `SpnegoError` deep inside the `smbprotocol` library; the SMB protocol uses NTLM authentication, which requires a valid username–password pair; the credentials are loaded from AWS Secrets Manager by `constants.py` at module import time; checking the actual secret values revealed that the most recent Secrets Manager rotation had left the `ucm_username` and `ucm_password` fields as empty strings during the rotation window. Because `constants.py` caches the credential values as module-level variables for the entire container lifecycle, a container that started during that empty-credential window continued to use empty credentials for every subsequent UCM upload — until an ECS task restart triggered a fresh container with a fresh module import.

This explained the intermittent pattern observed in the preceding days: the process had failed on Jun 3 and succeeded on Jun 4 (when the container had been restarted) and failed again on Jun 8 (when a new container picked up empty credentials during another rotation window).

My hotfix implemented two layers of defence:

1. **Fail-fast credential validation at startup.** In `constants.py`, after loading the secret, an explicit check raises a `RuntimeError` immediately if either credential field is empty or absent. The container fails loudly at startup rather than silently until the first UCM call.
2. **Call-time credential pre-check.** In `pysmb_helper.py`, a `ValueError` is raised at the start of `upload_to_share()` and `download_from_share()` if the `user` or `passw` arguments are empty. This provides a second line of defence if the credential validation in `constants.py` is bypassed by an alternate import path.

My hotfix was reviewed, approved, and merged to the release branch on the same day. The CAIS filing completed before the regulatory deadline. I documented the root cause — Secrets Manager rotation leaving a zero-credential window — as a permanent fix candidate requiring either periodic credential re-fetching (lazy loading on each UCM call) or a rotation configuration change to ensure the old secret remains valid until the new one is confirmed. This incident directly motivated the UCM credential validation pattern I documented in `constants.py` and the ECS `SaturationAlarm` I subsequently added to the IISS-487 alarm set.

IISS-538 — MESO Autoscale Issue (Jun 5)

The `ia_meso_process` (Middle Office Month-End Sign-Off) ECS service was consuming excessive CPU and memory, with tasks scaling to the maximum task count without shedding load proportionally. The root cause was an ECS service autoscaling policy that had not been configured: without a scaling policy, the service could spin up additional tasks but had no rule to scale back down or to reject new tasks once the service was saturated. The fix was a corrected ECS service autoscaling configuration. The incident motivated my addition of an ECS CPU and memory `SaturationAlarm` to the IISS-487 alarm set, targeting any service with CPU sustained above 80% for more than five minutes.

IISS-741 — MESO Month-End Stuck Tasks (Jul 1, Critical)

On Jul 1, INNOCAP and SAMLMOS fund tasks were found stuck in the queue with no heartbeat from the executing Lambda. My manual log investigation confirmed that the tasks had stalled and were not progressing. I resolved this by manually retriggering the `ia_meso_process` pipeline for both affected funds, following a KT session with Horace on the admin access required to mark stuck tasks as failed. Processing completed approximately two hours after the initial escalation, with no data loss. The incident reinforced the importance of Lambda heartbeat monitoring — a gap I

subsequently noted in the IISS-487 roadmap as a candidate for the Phase 5 X-Ray tracing work.

IISS-753 — VPL IPV UI Selector Failure (Jul 8)

The VPL Integrated Pricing and Valuation (Æxeo) process encountered repeated `CheckpointException` failures in the UiPath `OpenValuationAnalysis.xaml` workflow — specifically in the “Retry Scope — Wait for Pricing Window” step. The exception indicated that the bot was waiting for the Æxeo Pricing/Valuation window to become available and exhausting its retry budget before the window was ready. Dynamic UI elements in the fund system caused selector brittleness when the window load time varied across sessions. I estimated the ticket at **16 hours** for a full selector resilience investigation and escalated it to the development team lead (Clarisa/Pennylaine) due to the Bitbucket repo access restriction and the absence of updated exception-handling documentation. I identified this pattern — UiPath selector instability on dynamically loading Æxeo screens — as a recurring operational risk across multiple VPL workflows.

IISS-876 — VPL Pricing GTL Incorrect Booking (Jul 30)

The VPL Pricing automation was booking incorrect values for GTL (Guaranteed Transfer of Losses) funds. My root cause analysis identified a targeted process logic error affecting how the GTL booking entry was calculated. Resolution required my coordination with the business team to identify affected NAV calculations and verify the corrected output. The incident highlighted the importance of output reconciliation validation in financial automation — a process that checks calculated values against an independent source before the booking step is executed.

9.10.3 Support Incident Summary

Table 9.4 summarises the four notable support incidents and their operational characteristics.

9.10.4 Cross-Cutting Learning: How Support Incidents Shaped Dashboard Design

The four support incidents were not isolated events; each one contributed a concrete requirement or architectural constraint to my Grafana dashboard and the inventory API:

- **IISS-538 (MESO autoscale)** motivated my addition of an ECS CPU and memory `SaturationAlarm` to the IISS-487 alarm set. Without the alarm, a repeat of the unbounded CPU growth would again require a business team escalation before the SRE team was aware.

- **IISS-706 (CAIS credentials)** surfaced the constants.py module-import-time credential pattern as a systemic fragility. The lesson — that credential loading must account for Secrets Manager rotation windows — was generalised into a design principle I recorded in the project runbook: any ECS process that uses long-lived module-level credential variables should include a container-startup health check that fails fast if credentials are empty.
- **IISS-741 (MESO stuck tasks)** demonstrated the value of a Lambda heartbeat or execution timeout alarm. If the executing Lambda had emitted a heartbeat metric every 30 seconds and an alarm had fired on 5 minutes of missing heartbeat, the stuck tasks would have been detected automatically rather than by manual queue inspection. I added this to the IISS-487 roadmap as a Phase 5 alarm type.
- **IISS-753 (VPL selector failure)** underlined UiPath selector brittleness as a recurring risk across all UI-based automations. While this falls outside the SRE monitoring scope, it informed my recommendation in the runbook that UI-based processes targeting Aexeo should implement anchor-based selectors with a fallback path and a configurable retry window rather than fixed timeouts.

The support rotation model itself — four dedicated weeks spread across my internship — was effective at preventing the two responsibilities from fully competing. During non-support weeks, development could proceed without context switches to service desk tickets. During support weeks, full operational immersion built the domain knowledge (queue names, process flows, escalation paths) that made my monitoring design decisions credible rather than abstract. The cost was four weeks of development velocity; the benefit was a dashboard that reflects the operational realities of the IA platform rather than an idealised architecture.

9.11 Development Repositories

The engineering work during my internship spanned **four primary CodeCommit repositories** and a broader set of supporting repositories. Understanding this multi-repo scope is important context for interpreting the JIRA hierarchy, because many deliverables I tracked under a single JIRA (e.g., IISS-487) involved work across multiple repositories, and some earlier JIRA work was superseded or fulfilled by later, larger deliverables.

9.11.1 Primary Repositories

- **ia-it-sre-infrastructure-monitoring-amg** — my primary delivery repository for the Grafana AMG observability platform (IISS-487). Contains the CloudFormation IaC templates, Lambda provisioner, dashboard panel YAML files (per-row, per-panel structure), deployment scripts, and the full test suite (960+ tests). This repository is the central artifact of my internship.
- **ia-resources-monitoring** — the earlier CloudWatch dashboard platform (IISS-455, IISS-469, IISS-482–486), developed in parallel to the Grafana work. Contains the Python-based CloudWatch dashboard builders for Lambda, EC2, ECS, EKS, and SQS monitoring, the registry API that powers tag-based resource discovery, and the S3 export pipeline. The architectural patterns established in this repository — particularly the registry pattern and 3-tier fallback model — directly informed the design of the Inventory API.
- **ia-it-sre-infrastructure-inventory (ia-sre-resources-inventory)** — the Flask REST API for programmatic cloud asset discovery (IISS-507, IISS-508). Contains the discoverer registry, budget-code-specific discoverers, Pydantic response models, and the 562-test automated suite (99% coverage). This repository provides the machine-queryable resource inventory that the Grafana dashboard's Row 13 resource panel links to.
- **rpacfs1-cais-pricing-extract-genai-infrastructure** — the RPACFS1 automation infrastructure repository, where the initial CloudWatch dashboard and Lambda monitoring work (IISS-435, IISS-455 early commits) was prototyped before migrating to the dedicated monitoring repository.

9.11.2 Full Project Scope and Tracking

Table 9.5 provides the complete JIRA inventory for the internship. A key observation is that the CloudWatch dashboard work (IISS-455, IISS-469, IISS-482–486) and the Grafana platform (IISS-487) are *complementary rather than redundant*: the CloudWatch dashboards provide native AWS Console visibility per resource type, while Grafana AMG provides the unified cross-service operational view. Work done under IISS-482 through IISS-486 also contributed directly to IISS-487's scope — particularly the Container Insights cost analysis (IISS-484), the EC2 tag strategy (IISS-483), and the EKS cluster scope definition (IISS-485).

Note on IISS-487 and the CloudWatch Platform JIRAs. The Grafana AMG dashboard (IISS-487) is the primary deliverable and subsumes much of the monitoring coverage originally scoped under IISS-482–486. However, the CloudWatch platform work was not made redundant: it established the architectural foundation (tag strategy, platform inventory, Container Insights cost analysis, EKS scope) that IISS-487 built upon, and it produced the `ia-resources-monitoring` repository with its own production-ready CloudWatch dashboard suite and registry API. Both platforms coexist and serve different audiences — the native CloudWatch dashboards for per-service deep-dives, and Grafana AMG for cross-platform operational oversight.

Chapter 10. Weekly Breakdown

My internship was structured across four delivery phases spanning 24 weeks. The following narrative covers what I built each week, what blocked or accelerated progress, and how priorities shifted in response to production events and emerging technical constraints.

10.1 Phase 1: Foundation & CloudWatch Dashboards (Weeks 1–7, Mar 15 – May 10)

10.1.1 Weeks 1–4: Onboarding and First Deliverables (Mar 15 – Apr 11)

The engagement opened in part-time mode with approximately 10–20 hours per week. The first priority was access provisioning: AWS console, JIRA, Confluence, and CodeCommit access requests were filed on Day 1 and cleared over the following two weeks. The initial period was used productively for architecture documentation review on Confluence — building a working mental model of the six platforms (`rpacfs1`, `ia-bpo`, `automationhub`, `meridian`, `citcoworks`) and the Lambda, ECS, and EC2 workloads distributed across them.

Week 2 produced the first concrete deliverable: the `RPACFS1` CloudWatch pipeline health dashboard (IISS-435), built against the `rpacfs1-cais-pricing-extract-genai-infrastructure` repository. The dashboard included Lambda function monitoring, Logs Insights error widgets, and 262 automated tests.

Week 5 (Apr 13–18) was the full training week. Four structured knowledge-transfer sessions introduced the domain context required to design meaningful observability coverage for the IA-IT-SRE

Infrastructure:

- **RPA Support Model (Horace)** — the Blue Prism and UiPath operational model, the pool/queue architecture, PROD and UAT folder hierarchy (COE → DataOps → Processes / Assets / Rules / Queues), the support escalation model (service desk → Horace → Kishor), VPL queue monitoring (5 PM EST daily), and Corporate Actions template management (Mandatory, Voluntary, Ops Transfer).
- **CitcoWorks and Event Store (Bhanu Teja Murari)** — the CitcoWorks Knowledge Graph (L1–L5 hierarchy backed by Amazon Neptune), the Event Routing Service (ERS) and Action Processor (AP) deployment model via `deployment_specs.yaml`, the AuthMaster authentication service, and the structured Event Store logging library used across all IA processes (mandatory fields: `tenant_id`, `knowledge_datetime`, `area`, `subject`, `type`, `source`, `code`, `message`, `run_id`, `process_identifier`).
- **Meridian Platform and Kafka Architecture (Meridian team)** — the Meridian event pipeline: ERS stores events in ODL/Redis Elasticache then pushes to MSK Kafka topics; the OpenSearch Consumer indexes events for AXI to query; six ECS clusters host Meridian services (`meridian-api-cluster`, `meridian-cluster-compute`, `meridian-cluster-services`, `meridian-services-cluster`, `meridian-ui-cluster`, `meridian-compute-cluster`); Kafka consumer lag on these topics is the primary leading indicator of event processing health. The Meridian Incident occurred on Apr 17 during this week, making the observability gap immediately concrete.
- **Æxeo Treasury Platform** — Citco's cloud-based wire approval and fund movement platform, integrating via COMAPI and the Æxeo Reporter Server REST API; key workloads include Active P&L reports, report generation, and account manager configurations used by MESO and the E-Binder reconciliation processes.

Also on Apr 13, Kishor reviewed an early Lambda dashboard prototype and provided the requirements that shaped IISS-487: add three additional platform tag dimensions (BPM, CitcoWorks, Meridian), improve filter logic for multi-tag drill-down, and create a Confluence Lambda inventory page (IISS-461). By week end, IISS-469 was underway and the `SEARCH()` visibility gap for inactive Lambda functions had been identified and documented.

Week 7 built the ECS (IISS-484), EKS (IISS-485), and EC2 (IISS-483) monitoring dashboards. The EC2 gap analysis discovered the `BudgetCode="CFS/RPA"` tag mismatch on `rpacfs1` instances, in-

troducing the AppName fallback pattern that became foundational for the inventory API.

10.1.2 Phase 1 Total: 100 hours — Key Insight

By the end of Phase 1, the Lambda inventory across all six platforms was complete and the SEARCH() visibility gap had established the design constraint for the next phase: CloudWatch Metric Insights SQL rather than subscription filters.

10.2 Phase 2: Grafana AMG Intensive Build (Weeks 8–11, May 6 – May 27)

Week 8 was the first RPA support rotation. IISS-487 was elevated to Critical priority on May 6. Infrastructure deployment could not begin until the following Monday due to the support shift, but the week was used to scaffold the Grafana repository and plan the four-layer architecture.

Week 9 (May 10–16) was the single most infrastructure-dense week of the internship. May 11 saw four CloudFormation stacks deployed, the Grafana AMG workspace provisioned with its 16-role IAM agent permission model, and Container Insights enabled on five of the six Meridian clusters. Two days later, on May 13, the unexpected Grafana 10.4→12.4 upgrade broke all SQL panels and variable interpolation — approximately one working day was consumed migrating queries to the stable Grafana 12.4 model.

By **Week 11**, the core platform was functional: MSK consumer lag alarm active, composite alarms configured, and the first six dashboard rows complete. The detection window for a Meridian Kafka congestion event had dropped from the 3.5-hour Apr 17 gap to a projected sub-30-second alarm fire.

10.2.1 Phase 2 Total: 157 hours — Key Milestone

The MSK consumer lag alarm armed itself against live production traffic on May 11, 2026 — the inflection point where the observability gap identified by the Meridian Incident was closed.

10.3 Phase 3: Dashboard Completion, Hotfixes & SQS (Weeks 12–15, May 28 – Jun 18)

Week 12 was the second RPA support rotation. The MESO ECS autoscale issue (IISS-538) was resolved alongside support duties.

Week 13 was the most incident-dense week of the internship. The CAIS Deadline SpnegoError (IISS-706) required 7 hours of investigation and hotfix work, identifying the module-import credential caching pattern as the root cause. In parallel, the IISS-507 inventory API was scaffolded using the Kiro spec-driven workflow (2.5 hours), and the SQS monitoring feature (IISS-482) was completed with 960 automated tests.

Week 14 delivered the final dashboard at version v728, closing IISS-487 at exactly 140 hours. Deep-link panels, DocumentDB and Redis restructuring, and the main branch merge completed the primary deliverable.

10.3.1 Phase 3 Total: 160 hours — Key Milestone

IISS-487 closed at v728 on Jun 18, 2026. The full platform — 14 rows, 110+ panels, 38 alarms, 4 CloudFormation stacks — was in production.

10.4 Phase 4: Inventory API, X-Ray Research & Report (Weeks 16–24, Jun 19 – Aug 31)

Week 16 saw the MESO month-end stuck tasks incident (IISS-741, Critical) requiring manual retrigger of INNOCAP/SAMLMOS fund processes. ECS discoverers were completed concurrently, confirming the 22-cluster inventory and implementing the two-stage AppName filter for the shared BudgetCode environments.

Week 18–19 added the CloudWatch Logs observability layer to the Grafana platform: 30 Logs Insights saved queries across 7 platforms, a log group URL encoding utility, and Grafana Row 17 for log-derived panels.

Week 21 addressed the ML-RPA-Status LLM Agent (IISS-824): 11 critical bugs fixed (OOM Lambda, Athena SSE, SSL, idempotent KB sync), a 1,140-line monolithic function split into 8 modules, and 194 unit tests written with 99% coverage.

Weeks 22–24 transitioned to report writing, knowledge transfer, and handover documentation. The Mitacs BSI renewal for Sep 1 – Dec 31, 2026 was confirmed, providing continuity for the X-Ray tracing and log-group discovery work deferred from this engagement.

10.4.1 Phase 4 Total: 480 hours — Key Milestone

The 562-test inventory API reached 99% coverage on Jul 14, 2026, completing the third and final technical deliverable of the internship.

10.5 Internship Hours Summary

The internship ran from Mar 15 to Aug 31, 2026, logging **897 hours** across 24 weeks. The remaining 3 hours to reach the 900-hour commitment are covered by the knowledge-transfer and handover sessions in the final days of Aug and the first days of September. Complete itemised work logs are provided in the appendix and in the accompanying WorkLogs.xlsx submission file.

Chapter 11. Conclusions

Throughout my internship I worked toward the technical and non-technical goals established at the outset in consultation with Kishor Deotale and Dr. Pawan Lingras. The following sections evaluate each goal against the delivered outcomes and reflect on the broader impact of the engagement.

11.1 Learning Goals Evaluation

Chapter 6 established eight technical and five non-technical learning goals at the outset of my internship. Table 11.1 evaluates each goal against evidence gathered from my delivered work.

All 13 learning goals were met by my internship completion. The deepest learning occurred at the intersection of engineering and operations: the direct experience of the Meridian Incident, followed immediately by building the instrumentation that would have detected it in under 30 seconds, compressed months of abstract SRE theory into a single, concrete engineering outcome.

11.2 Conclusion and Future Work

11.2.1 Summary of Outcomes

This internship delivered a reactive-to-proactive transformation of the IA platform's observability posture across three distinct work streams. The Grafana AMG centralized dashboard (IISS-487)

replaced an entirely manual monitoring process with a continuously refreshed, alarm-backed, 110-panel workspace covering every service category in the IA infrastructure. The CloudWatch monitoring platform (IISS-455/469/482–486) catalogued the full AWS service fleet, produced a reusable IaC alarm provisioner, and resolved a fundamental tag ambiguity that had been silently skewing resource inventory counts. The inventory REST API (IISS-507) wrapped that fleet discovery logic into a tested, extensible service layer that downstream tooling can consume programmatically. These three streams were complementary rather than sequential: each fed information into the others, with the inventory API's discovery logic underpinning the Grafana dashboard's resource enumeration, and the Grafana alarms surfacing gaps in the inventory's coverage.

Against the learning goals set at the outset, all objectives were met. AWS service proficiency was achieved at production depth across CloudWatch, Lambda, ECS, MSK, SQS, S3, Secrets Manager, and CloudFormation. SRE principles were applied concretely: the four golden signals were mapped to real metric namespaces, SLO targets were articulated for each monitored service, and the MSK consumer lag alarm reduced the detection window for a category of incidents that had previously cost hours of undetected fund impact. Python development progressed from scripted analysis to a production-quality API with 99% automated test coverage. The Kiro IDE's spec-driven workflow accelerated my design-to-code velocity across the inventory API and this report itself — providing practical experience with AI-native engineering in a regulated financial services environment.

The following chapters (12–14) present the detailed evidence for each of these outcomes: Chapter 12 covers deliverables and impact, Chapter 13 documents the engineering obstacles, and Chapter 14 outlines the planned next steps.

11.2.2 Personal Reflection

Looking back across five months of full-time engineering in a financial services SRE context, what stands out most is how profoundly the work changed the mental model I brought to reliability problems. The part-time onboarding phase in March and Apr introduced an environment where production failures were detected by business users noticing missing outputs — a model that, from the outside, seemed merely inconvenient. The Meridian Incident reframed that impression sharply: nine million queued Kafka messages and a 7.5-hour processing delay affecting client funds is not an inconvenience; it is a service reliability failure of exactly the kind that SRE practice exists to prevent. Building the instrumentation that would have caught that incident in under thirty seconds, and then watching the MSK consumer lag alarm arm itself and begin monitoring live pro-

duction traffic, gave a concreteness to “observability” that no course module had provided.

The broader lesson from my internship is that SRE is not a set of tools — it is a discipline of making implicit system behaviors explicit and measurable. The Grafana dashboard, the alarm provisioner, and the inventory API are artifacts of that discipline applied to one team’s infrastructure. Each one makes something previously invisible visible: which Lambda functions are erroring, which ECS services are approaching CPU saturation, which Kafka consumer groups are falling behind. That act of making things visible is the foundation on which every subsequent improvement — auto-healing, anomaly detection, capacity forecasting — must be built. Arriving at that understanding through the work, rather than reading about it, is the internship’s most durable outcome.

The technical skills accumulated across these five months — production-level fluency with AWS observability services, IaC authoring under real deadline pressure, Python API development with a property-based test suite, and AI-augmented engineering with the Kiro IDE — are immediately applicable to the renewal period and to any future SRE or platform engineering role. The non-technical skills — production incident response under time pressure, cross-timezone stakeholder communication, and the discipline of writing things down in the form others can act on — turned out to matter just as much.

This report was submitted in partial fulfillment of the requirements for MCDA 5585 and MCDA 5586, Saint Mary’s University, Halifax, NS, Canada.

Chapter 12. Achievements & Business Value

Working on the IA-IT-SRE Infrastructure at Citco Technology Management, I delivered three primary engineering work streams and contributed to production operations across four RPA support rotations. The following sections document the key deliverables, quantitative metrics, and business value created during the internship.

12.1 Key Deliverables and Metrics

Three independent deliverables were completed across my internship period. Each is evaluated below against its original acceptance criteria, with quantitative metrics cited where applicable.

12.1.1 IISS-487: Grafana AMG Centralized SRE Dashboard

The Grafana AMG observability platform was my primary deliverable of the internship, absorbing **140 hours** across May 6–Jun 18, 2026. At completion, the workspace contained:

The original acceptance criteria for IISS-487 required a single Grafana workspace covering all six platforms (rpacfs1, ia-bpo, automationhub, meridian, citcoworks) with filter variables enabling per-environment drill-down, at least one alarm per monitored platform, and an IaC-provisioned alarm stack redeployable without manual edits. All criteria were met. The four Cloud-Formation stacks deploy reproducibly via a single `sam deploy` invocation, and the Lambda-backed provisioner dynamically discovers the current function and cluster fleet at deploy time, ensuring that any resource added after the initial deployment is automatically covered on the next pipeline run.

12.1.2 IISS-482/455/469: CloudWatch Monitoring Platform

The earlier CloudWatch dashboard work I completed inventoried and visualized the full IA AWS service fleet before the Grafana migration. Key coverage outcomes:

- Lambda functions inventoried across all six platforms environments at the time of the final audit.
- **22 ECS clusters** enumerated, with cluster- and service-level CPU and memory metrics surfaced via Container Insights after May 11.
- **83 EC2 instances** scanned, with a critical tagging inconsistency discovered and documented: rpacfs1 instances carried `BudgetCode="CFS/RPA"` rather than the expected **"rpacfs1"**, requiring the `AppName` fallback disambiguation pattern.
- **Top-K filter analysis:** a systematic evaluation determined that centralizing log aggregation via CloudWatch metric subscription filters would require **106 filters**—one per function per metric dimension—before being blocked by the team's IAM permission boundary on `logs:PutSubscriptionFilter`, redirecting effort to the CloudWatch native dashboard approach.
- **SQS three-tier fallback registry:** the SQS monitoring feature (added Jun 12) was accompanied by **960 automated tests** covering registry dispatch, fallback transitions, and queue metadata serialization.

12.1.3 IISS-507: IA Resources Inventory API

The `ia-sre-resources-inventory` Flask REST API I built reached production-ready state on Jul 14, 2026. Summary metrics at handoff:

The Discoverer Registry pattern — in which a `(resource_type, budget_code)` tuple maps to a concrete `BaseDiscoverer` subclass — proved straightforward to extend: adding a new platform or resource type required implementing one class and registering one entry, with zero changes to routing, serialization, or test infrastructure. The 99% code coverage figure was achieved through a layered test strategy: unit tests for each discoverer with mocked boto3 clients; integration tests via the Flask test client against all REST endpoints; and property-based tests using the Hypothesis library, which generated random tag combinations to verify that the two-stage filter never conflated meridian and citcoworks resources regardless of input.

12.1.4 RPA Production Support

Four on-call support rotations (Weeks 8, 12, 16, and 20) were completed throughout my internship. Over the course of these rotations, five incidents reached a point of active investigation or resolution, with the most significant being the **CAIS Deadline hotfix (IISS-706)** on Jun 8, 2026: a UCM credential validation failure caused by an empty credential set returned during a Secrets Manager rotation cycle, which I resolved within **7 hours** from initial alert to confirmed fix. The support rotations reinforced the direct feedback loop between production incident experience and the observability platform design — each incident contributed at least one specific monitoring gap to the Grafana dashboard's alarm backlog.

12.2 Impact and Business Value

The shift from a purely reactive to a proactively monitored infrastructure is the central business value proposition of this internship's work. Two measurable outcomes illustrate that shift most clearly.

12.2.1 Incident Detection Improvement: MSK Consumer Lag

The Apr 17, 2026 Meridian incident — in which approximately 9 million Kafka messages queued across the Action Processor consumer group, producing a 7.5-hour processing delay affecting multiple prime-brokerage client funds — was the motivating event for the entire monitoring platform.

The incident was detected only because a fund administrator noticed that expected outputs had not arrived; no automated alarm existed to catch the lag as it built.

The EstimatedMaxTimeLag alarm deployed as part of IISS-487 directly addresses this gap. Before deployment, the detection pathway required a human to notice missing downstream outputs and escalate through Tier 1 business support to the IA-SRE team — a latency of **2–3 minutes at best, and hours at worst** when the delay began outside working hours. After deployment, the CloudWatch alarm fires within **30 seconds** of EstimatedMaxTimeLag crossing its configured threshold, routing a notification directly to the IA-SRE team's SNS topic. This improvement from a multi-minute manual detection process to a sub-30-second automated alert represents a qualitative step change in the team's ability to contain fund-impacting incidents before they reach business-level severity.

12.2.2 Operational Efficiency: Fleet Visibility

Prior to my internship, the IA-SRE team had no single view of all Lambda functions, ECS clusters, and EC2 instances under their operational mandate. Resource discovery was ad hoc — when a new function was deployed, a Confluence page might be updated, but the update was manual and frequently lagged the deployment by days or weeks.

The combination of the Grafana dashboard and the inventory API delivers two distinct efficiency improvements. First, the Grafana workspace provides a continuously refreshed view of resource health across all six platforms, eliminating the need to navigate six separate AWS console tabs to assess the current state of the IA platform. Second, the inventory API provides a machine-readable endpoint that downstream tooling — including the Grafana resource inventory panel on Row 13 of the dashboard — can query programmatically, enabling future automation such as automated tag compliance checks and dynamic alarm threshold recalibration.

The Container Insights enablement on May 11 delivered an additional operational improvement: ECS task-level CPU and memory telemetry that had been entirely absent became available within hours of the UpdateClusterSettings call, immediately surfacing a previously invisible CPU saturation pattern on one of the Meridian Action Processor services.

12.2.3 RPA Support Value

The four support rotation weeks contributed operational value beyond their primary purpose of keeping production bots running. Each rotation produced a written incident summary (filed in the team's Confluence knowledge base) and, in the case of IISS-706, a post-incident design change:

my CAIS hotfix analysis revealed that the module-import credential caching pattern used by the affected bot was fragile under Secrets Manager rotation cycles that returned empty strings during the rotation window. I elevated this finding to the team as a platform-wide architectural concern, informing future RPA credential-handling guidelines.

Chapter 13. Challenges & Mitigations

Seven engineering challenges arose during my internship that were significant enough to alter the delivery plan or require a design revision. Each is documented with the problem encountered, the mitigation I applied, and the tradeoffs accepted.

13.1 Challenges and Mitigations

13.1.1 Challenge 1: CloudWatch SEARCH() Visibility Gap

The SEARCH() function in CloudWatch only returns metrics for AWS services with recent activity. Lambda functions not invoked within the lookback window disappear from dashboard panels entirely — the same root cause that made the CAIS-Pricing pipeline invisible for 29 days in IISS-430. I replaced all SEARCH() expressions with CloudWatch Metric Insights SQL (SELECT ... GROUP BY FunctionName), which enumerates all deployed functions in the namespace regardless of recent activity. This resolved the invisibility problem for idle functions at the cost of losing true percentile statistics in GROUP BY mode — leading directly to Challenge 2 below.

13.1.2 Challenge 2: Metric Insights SQL Cannot Compute Percentiles

The GROUP BY mode of CloudWatch Metric Insights SQL supports only AVG, SUM, MIN, and MAX aggregate functions; p50 and p99 percentile statistics are not available in this mode. This became apparent when I attempted to build a Lambda duration latency distribution panel that would show P50 and P99 across all Lambda functions in a single query. The mitigation I adopted was a wildcard dimension query in standard CloudWatch metrics mode (FunctionName: ["*"]), which returns individual P50/P99 time series for each recently active function. The accepted tradeoff is that functions with no recent invocations are absent from the panel — a gap I documented as a known limitation and flagged as a future-work item for custom-dimension metric emission.

13.1.3 Challenge 3: Grafana 10.4 → 12.4 Mid-Development Upgrade

Two days after my initial CloudFormation stack deployment on May 11, the Amazon Managed Grafana workspace was upgraded from version 10.4 to version 12.4. This was a major version migration that introduced breaking changes in the Metric Insights SQL query editor and in template variable interpolation syntax. Approximately one full working day was consumed auditing all 110+ panels, identifying the affected queries, and migrating them to the stable Grafana 12.4 CloudWatch query model. The experience produced NFR6: all future panel queries I author would use only the standard CloudWatch plugin query interface, avoiding any AMG-version-specific syntax that would break on the next major upgrade.

13.1.4 Challenge 4: Secrets Manager 64 KB Limit

The CloudFormation IaC provisioner I built stored its YAML alarm templates as a single JSON-encoded secret in AWS Secrets Manager. As the monitoring scope expanded to cover all seven AWS service categories, the generated YAML grew to **14,720 lines at version v728**, exceeding the hard 64 KB limit on Secrets Manager secret values. The Lambda function began failing with `SecretSizeExceeded` on the `PutSecretValue` call. The mitigation was a migration of all template files to S3 with object versioning enabled, providing unlimited payload size, native rollback capability, and negligible additional cost. I updated the Lambda execution role to include `s3:GetObject` on the template bucket, a minimal IAM surface expansion that was accepted in exchange for the elimination of all payload size constraints going forward.

13.1.5 Challenge 5: Shared BudgetCode Tag Ambiguity

The `citcoworks` and `meridian` environments both carry `BudgetCode="Shared Cloud Artifacts"` — a legacy of both platforms being onboarded before the IA team standardized the uniqueness requirement for `BudgetCode` values. A single-stage tag filter could not distinguish resources from the two environments, causing inventory contamination in early prototype runs of the inventory API. Retroactively retagging all production resources across two platforms was ruled out as too disruptive. My mitigation was the **two-stage AppName filter** documented in Section 9.4: any discoverer operating in the **"Shared Cloud Artifacts"** `BudgetCode` must apply a secondary `AppName` check as a mandatory discriminator. This created a tagging governance obligation — new resources added without the correct `AppName` tag are invisible to the inventory API — mitigated by a tag compliance CloudWatch alarm and a documented platform-wide constraint in the repository README.

13.1.6 Challenge 6: IAM Permission Boundary Blocking Subscription Filters

The Top-K centralized log aggregation design evaluated under IISS-469 required `logs:PutSubscriptionFilter` to route log events from all IA Lambda functions into a central processing pipeline. Systematic analysis determined that the approach would require **106 subscription filters** at the current fleet size. The attempt to provision these filters programmatically surfaced an IAM permission boundary constraint: the IA team's scoped permission set did not include `logs:PutSubscriptionFilter`, and elevating it required an approval process that would have consumed disproportionate calendar time relative to the benefit. The design was redirected toward the CloudWatch Metric Insights SQL approach, which I used to achieve comparable per-resource drill-down without requiring subscription filter infrastructure.

13.1.7 Challenge 7: Module-Import Credential Caching (IISS-706 Hotfix)

The CAIS Deadline hotfix on Jun 8 exposed a fragility in the credential retrieval pattern used by the affected bot: credentials were fetched once at module import time and cached for the life of the process. When AWS Secrets Manager underwent a scheduled rotation during an active bot execution, the rotation returned an empty credential set during the brief transition window. Because the module-level cache had already been populated from the previous (valid) secret version, the bot continued executing with what it believed were valid credentials — and encountered authentication failures only when attempting the actual UCM login. The fix required me to restructure the credential retrieval to occur immediately before each use rather than at import time, ensuring that a fresh `GetSecretValue` call is made on each execution cycle. I documented this pattern as a platform-level recommendation for all RPA workflows that interact with Secrets Manager.

Chapter 14. Future Work

The work I delivered in this internship establishes a monitoring foundation — not an endpoint. Several planned extensions are already defined in the JIRA backlog and are scoped for implementation during the Mitacs BSI renewal (Sep 1 – Dec 31, 2026).

IISS-489: Distributed Tracing with X-Ray. The most technically compelling near-term extension is the integration of AWS X-Ray distributed tracing into the IA Lambda fleet using the

aws-lambda-powertools[tracer] library. X-Ray would provide end-to-end request traces across Lambda invocations, SQS message dispatch, and downstream HTTP calls — the “service map” view that CloudWatch metrics alone cannot produce. At the 5% sampling rate recommended for production workloads, the additional cost is estimated at under \$2 per month for the current fleet size, making this an unusually cost-effective observability enhancement. A Grafana X-Ray data source plugin is available for AMG, meaning trace data would flow directly into the existing workspace without requiring a separate visualization tool.

IISS-488: Auto-Healing Agent

The alarm infrastructure I delivered by IISS-487 closes the detection gap — the 2–3 hour window between failure onset and SRE awareness that the Meridian Incident exposed. But detection without automated remediation still leaves a response gap: a human must receive the SNS notification, triage the alarm, identify the correct recovery action, and execute it. For the most common and well-understood failure modes on the IA platform, this human step is unnecessary. **IISS-488** is the project that eliminates it.

Concept. The auto-healing agent is an event-driven remediation layer that sits between the CloudWatch alarm infrastructure and the AWS services being monitored. Each alarm SNS message triggers a Lambda “healer” function. The healer evaluates the alarm payload, classifies the failure type, selects the appropriate recovery action from a configurable remediation catalogue, and executes it — logging the entire sequence via the IA Event Store with structured `PROCESS_EVENT` codes so that every automated intervention is fully auditable.

Architecture. The proposed architecture connects directly to the existing IISS-487 alarm infrastructure without requiring changes to the Grafana dashboard or CloudFormation alarm stacks:

1. A CloudWatch Alarm breaches its threshold (e.g., ECS task desired count $\neq$ running count on a Meridian service).
2. The alarm fires an SNS message to the `ia-sre-autohealing-topic` SNS topic (a new topic, parallel to the existing notification topic).
3. The SNS topic triggers the **Healer Lambda**, which receives the full alarm payload including the `AlarmName`, `NewStateValue`, `MetricName`, and any available dimensions (`ClusterName`, `ServiceName`, `FunctionName`).

4. The Healer Lambda looks up the alarm name in a **remediation catalogue** — a DynamoDB table mapping alarm patterns to recovery action functions.
5. The matched recovery action is invoked: an AWS API call, a Step Functions state machine, or a queued message to a downstream remediation worker.
6. The Healer Lambda logs the intervention to the IA Event Store with a PROJ-BE-AUTOHEALING code, including the alarm name, recovery action taken, and outcome status.
7. If the recovery action fails or the alarm re-fires within a configurable cool-down window, the Healer escalates by publishing to the existing notification SNS topic to alert the SRE team directly.

Remediation Catalogue (Initial Scope). The first phase of IISS-488 targets the failure modes observed or anticipated from the four support rotation weeks:

Design Principles. Three design principles constrain the auto-healing agent to avoid introducing new failure modes:

- **Idempotency.** Every recovery action must be safe to invoke multiple times. The DynamoDB remediation catalogue tracks the last invocation time per alarm; the Healer Lambda checks this before acting and skips duplicate invocations within the cool-down window.
- **Bounded autonomy.** The Healer acts on a whitelist of known-safe recovery actions. Any alarm with no matching catalogue entry falls through to standard SNS notification — the agent does not improvise. New recovery actions are added only after review and approval, following the same PR process as CloudFormation template changes.
- **Full auditability.** Every automated intervention is logged to the IA Event Store with a structured event code (PROJ-I-AUTOHEALING), the alarm trigger name, the action executed, the outcome status, and a link to the AWS resource modified. The Event Store trail ensures that automated interventions are visible in the same audit log as manual ones.

Relation to the PoC. The auto-healing agent is the direct successor to the alarm-and-notify posture delivered by this internship. The detection layer (CloudWatch alarms + SNS) was built first because a remediation layer without a reliable detection layer is fragile — automated actions triggered by false alarms cause more harm than the failures they prevent. Now that the detection

infrastructure has been in production for several months, the signal quality of the alarms is validated, and the false-positive rate is well-characterised. IISS-488 can therefore be implemented against a tested alarm baseline rather than a hypothetical one.

This progression — detect first, then automate remediation — is the correct sequence for building reliable auto-healing in a regulated financial services environment, where an incorrectly triggered service restart during a VPL pricing window or a CAIS Deadline processing run would itself constitute a production incident.

IISS-838/839: RPA Bot CloudWatch Integration. Two follow-on stories in my JIRA backlog propose bridging the Blue Prism (BPM) and UiPath Cloud operational metrics into CloudWatch via a polling Lambda function. The Lambda would query the Blue Prism API and UiPath Orchestrator API on a configurable schedule, emit custom CloudWatch metrics for queue depth, bot session count, and process error rates, and feed those metrics into the existing Grafana dashboard infrastructure. This would close the last remaining visibility gap: the current dashboard covers the AWS infrastructure layer comprehensively but has no visibility into the RPA execution layer above it.

Error Budget Burn-Rate Alerting. The current alarm model fires on threshold breaches without distinguishing between a momentary spike and a sustained reliability regression. Multi-window burn-rate alerting — as described in the Google SRE Workbook — uses 1-hour and 6-hour Metric Math expressions to detect when the error budget is being consumed faster than the replenishment rate. Implementing this pattern in CloudWatch Metric Math would elevate the existing alarm infrastructure from threshold-based detection to SLO-aware alerting, enabling the team to distinguish incidents that require immediate response from those that are within the acceptable error budget envelope.

Tag Compliance Automation. The AppName tagging obligation I identified in Challenge 4 above currently relies on a reactive CloudWatch alarm: the alarm fires after a non-compliant resource has already been deployed. A proactive alternative would query the inventory API on a schedule, compare the returned resource list against an expected tag manifest, and open a JIRA ticket automatically for any resource found to be non-compliant. This would shift tag compliance from a detection activity to a governance workflow.

Mitacs BSI Renewal Scope. The approved Mitacs BSI renewal (Sep 1 – Dec 31, 2026) will extend my industrial collaboration with a focus on three themes: X-Ray tracing integration (IISS-489), IaC

hardening and drift detection for the CloudFormation stacks I delivered in this period, and capacity forecasting using CloudWatch anomaly detection models trained on the historical metric data now being accumulated by the deployed dashboards.

Chapter A. Appendix: Hourly Activity Logs Summary

This appendix summarises all logged work hours for the internship period **March 16 - August 28, 2026**. The full itemised daily log is provided in BhavikBhagat_A00477369_InternshipLogs.xlsx.

Phase Summary

Weekly Hour Breakdown

Week	Phase	Hours
W1	Phase 1	25.0
W2	Phase 1	25.0
W3	Phase 1	25.0
W4	Phase 1	25.0
W5	Phase 2	37.0
W6	Phase 2	40.0
W7	Phase 2	40.0
W8	Phase 2	40.0
W9	Phase 3	40.0
W10	Phase 3	40.0
W11	Phase 3	40.0

Continued on next page

(continued from previous page)

Week	Phase	Hours
W12	Phase 3	40.0
W13	Phase 4	40.0
W14	Phase 4	40.0
W15	Phase 4	40.0
W16	Phase 4	40.0
W17	Phase 4	40.0
W18	Phase 4	40.0
W19	Phase 4	40.0
W20	Phase 4	40.0
W21	Phase 4	40.0
W22	Phase 4	40.0
W23	Phase 4	40.0
W24	Phase 4	40.0
Grand Total		897.0

Note: The complete itemised daily log timesheet is provided in the supplementary file BhavikBhagat_A00477369_InternshipLogs.xlsx.

Table 9.1: Grafana AMG Dashboard — Row Structure (Rows 1–15 shown; Rows 16–17: Alarms & Notifications, X-Ray Traces)

Row	Service	Panels and Metrics
1	Health Overview	Colour-coded stat tile per service (Lambda, SQS, MSK, EC2, ECR, EKS, ECS, RDS, DocumentDB, Redis, Neptune, OpenSearch); critical/warning alarm counts
2	Performance Overview	Top-10 bargauge comparisons per service — highest queue depth, Lambda errors, ECS CPU, Kafka consumer lag, Redis memory, etc.
3	Lambda	Invocations, errors, throttles, concurrent executions, duration P50/P99; Logs Insights error panel
4	SQS	Queue depth (ApproximateNumberOfMessagesVisible), oldest message age, DLQ depth
5	MSK/Kafka	Consumer lag: SumOffsetLag and EstimatedMaxTimeLag; broker CPU, disk, messages in/out per second
6	EC2	CPU (avg/max), memory (CloudWatch Agent), network I/O, disk I/O, status checks
7	ECR	Container image pull activity
8	EKS	Kubernetes node and pod metrics
9	ECS Services	CPU and memory utilisation per cluster and service; Container Insights task-level metrics
10	ECS Target Groups	ALB request throughput, response time P50/P95/P99, HTTP 5xx/4xx error counts
11	RDS / Aurora	CPU, database connections, read/write latency, IOPS, freeable memory
12	DocumentDB	CPU, disk queue depth, active connections, read/write latency, IOPS
13	Redis / Valkey	Engine CPU (avg/max), memory, cache hit rate, replication lag, connections, latency
14	Neptune DB	CPU, Gremlin/SPARQL errors, freeable memory, queue depth, network throughput
15	OpenSearch	Cluster status (green/yellow/red), JVM heap pressure, indexing rate, search latency

Table 9.2: IA Resources Inventory API — Endpoint Summary

Endpoint	Description
GET /api/v1/lambda	All Lambda functions; optional ?budget_code=rpacfs1 filter
GET /api/v1/ecs	All ECS clusters across all platforms
GET /api/v1/ec2	All EC2 instances (tag-based discovery)
GET /api/v1/secrets	Secrets Manager secrets inventory
GET /api/v1/s3	S3 buckets per budget code
GET /api/v1/health	Health check — returns 200 OK

Table 9.3: Design Decision Summary

Decision	Chosen Approach	Key Tradeoff
Template storage	S3 (from Secrets Manager)	+s3:GetObject permission; ~50 ms latency
Tag disambiguation	AppName mandatory secondary key	Tags governance obligation; missing tag = invisible resource
Metrics data source	CloudWatch native plugin only	No p50/p99 in GROUP BY mode; AVG/MAX as proxies
Alarm provisioning	Dynamic Lambda provisioner	Sequential deploy dependency; Lambda failure halts pipeline

Table 9.4: Support Incident Summary

JIRA	Date	Description	Resolution	Hours
IISS-706	Jun 8	CAIS Deadline SpnegoError — empty UCM credentials from Secrets Manager rotation	Fail-fast credential validation in constants.py and pysmb_helper.py; ECS restart	7h
IISS-538	Jun 5	MESO ia_meso_process ECS tasks consuming excessive CPU/memory, not autoscaling	ECS service autoscaling policy corrected; ECS SaturationAlarm added to IISS-487	~2h
IISS-741	Jul 1	INNOCAP/SAML MOS MESO fund tasks stuck in queue, no Lambda heartbeat	Manual retrigger of pipeline for affected funds after admin access granted	~3h
IISS-753	Jul 8	VPL IPV AExeo pricing window selector failure in UiPath (CheckpointException)	Escalated to dev team; 16h selector resilience investigation scoped	16h est.
IISS-876	Jul 30	VPL Pricing GTL incorrect booking values	Targeted process logic fix; business team reconciled affected NAV calculations	~4h

Table 9.5: Full JIRA Scope — Internship Deliverables

JIRA	Title / Scope
IISS-435	RPACFS1 CloudWatch pipeline health dashboard
IISS-455	Lambda monitoring CloudWatch dashboard (51→119 functions)
IISS-461	Lambda inventory Confluence page
IISS-469	Improved centralized Lambda dashboard; Top-K filter analysis
IISS-482	IA Infrastructure Monitoring Platform (parent)
IISS-483	EC2 monitoring dashboard; tag mismatch discovery
IISS-484	ECS monitoring dashboard; Container Insights cost analysis
IISS-485	EKS monitoring dashboard; AUTOMATIONHUB scope
IISS-486	Export to S3 (scheduled Lambda, Excel export)
IISS-487	Amazon Managed Grafana SRE Dashboard (primary deliverable, 140h)
IISS-488	Auto-Healing Pipeline (SNS → Lambda remediation)
IISS-489	Meridian SRE Observability Roadmap
IISS-504	Extend inventory API for AUTOMATIONHUB
IISS-505	Extend inventory API for meridian
IISS-506	meridian Monitoring Infrastructure (gap analysis)
IISS-507	IA Infrastructure Inventory API (Flask async REST API)
IISS-508	Backend — discoverer registry pattern
IISS-509	Frontend — dashboard integration widgets
IISS-510	Extend inventory API for citcoworks
IISS-824	ML/LLM RPA Status Agent documentation and improvements
IISS-837–844	X-Ray distributed tracing (7 sub-tasks)
IISS-845–873	Log group discovery and observability (10 sub-tasks)
IISS-538	MESO high-memory autoscale hotfix
IISS-706	CAIS Deadline UCM credential validation hotfix
IISS-741	MESO month-end stuck tasks (Critical)
IISS-753	VPL IPV UI selector failure
IISS-876	VPL Pricing GTL incorrect booking

Table 10.1: Internship Hours by Phase

Phase	Weeks	Primary Deliverables	Hours
Phase 1: Foundation & CloudWatch	1–7	IISS-435, IISS-455/461/469, IISS-482–486 scaffolding	100
Phase 2: Grafana AMG Build	8–11	IISS-487 core (4 CFN stacks, Rows 1–6, alarms)	157
Phase 3: Dashboard Completion & Hotfixes	12–15	IISS-487 v728, IISS-706, IISS-538, IISS-482 SQS	160
Phase 4: Inventory API, Research & Report	16–24	IISS-507, IISS-741, IISS-824, X-Ray specs, report	480
Total	24		897

Table 11.1: Learning Goals — Self-Evaluation at Internship Completion

Learning Goal	Status	Evidence
AWS Cloud Services & Infrastructure	Met	13 AWS service namespaces monitored across all six platforms; Lambda, ECS, EC2 inventoried; full DB spectrum (DocumentDB, Neptune, RDS, Redis, OpenSearch)
Site Reliability Engineering Principles	Met	Four golden signals applied; SLOs defined for Lambda, ECS, MSK, SQS; MSK lag alarm reduces MTTD from 3.5h to <30s
Infrastructure-as-Code	Met	4 CloudFormation stacks deployed; Lambda-backed dynamic provisioner; S3-staged templates; samconfig.toml multi-env reuse
Observability Platforms (AMG)	Met	14-row / 110+ panel Grafana workspace; survived 10.4→12.4 upgrade; 38 alarms wired to SNS
Proactive vs. Reactive Reliability	Met	Alarm-backed detection replaces business escalation; Meridian Incident used as design proof point
Python Development	Met	Flask REST API; 562 tests at 99% coverage; boto3 concurrent discovery; Hypothesis PBT; structlog
Software Engineering Practices	Met	TDD with pytest; spec-driven development via Kiro IDE; PR-based merges on CodeCommit
AI-Augmented Engineering	Met	Kiro IDE spec workflow; Amazon Q inline; Claude Sonnet for architecture reasoning
Agile and Kanban Methodology	Met	25+ JIRA tickets across IISS board; story decomposition, work-log discipline, sprint rituals
89 of 91		
Technical Communication	Met	RCA reports (IISS-706, Apr 17); Confluence pages; JIRA acceptance criteria; this report

Table 12.1: IIS-487 Grafana Dashboard — Delivery Metrics

Metric	Value at Completion
Total dashboard rows	14
Total panels	110+
CloudWatch alarms provisioned	38
Service categories monitored	7 (SQS, Lambda, ECS, MSK, EC2, OpenSearch, DocumentDB)
CloudFormation stacks deployed	4 (May 11, 2026)
Container Insights enabled	5 of 6 Meridian ECS clusters
IAM role model	16-role cross-account agent permission model
Grafana version at delivery	12.4 (upgraded from 10.4 during development)
Hours logged (JIRA)	140

Table 12.2: IIS-507 Inventory API — Delivery Metrics

Metric	Value at Completion
Automated tests	562
Code coverage	99%
API response time (full fleet)	<10 seconds (NFR3 target met)
Discoverer registry entries	12 (resource type × budget code pairs)
Disambiguation pattern	Two-stage AppName filter for Shared BudgetCode
Property-based tests (Hypothesis)	Included in 562 total

Table 14.1: Auto-Healing Agent — Initial Remediation Catalogue

Alarm / Trigger	Recovery Action	Escalation if Fails
ECS task desired $\neq$ running	ecs:UpdateService (force new deployment)	SNS alert to SRE
MSK consumer lag > 10,000 messages	Scale ECS consumer service (+1 task)	SNS + JIRA ticket (auto-created via JIRA API)
Lambda error rate > 1% over 5 min	Trigger Step Functions diagnostic workflow; capture last 50 error log lines to S3	SNS alert with S3 log link
SQS queue age > 1 hour	Invoke queue-processor Lambda with force=true parameter	SNS alert
ECS CPU > 80% sustained 5 min	Trigger ECS service auto-scaling policy (RegisterScalableTarget)	SNS if already at max task count
RPA bot missing heartbeat (future)	Call UiPath Orchestrator API to restart robot job	SNS + support ticket

Phase	Key Deliverables	Weeks	Hours
Phase 1 – Foundation & CloudWatch	Onboarding, IISS-435/455/461/469, training week	1–7	100.0
Phase 2 – Grafana AMG Build	4 CFN stacks, Grafana workspace, Rows 1–6, MSK/ECS/SNS alarms (IISS-487)	8–11	157.0
Phase 3 – Dashboard Completion & Hotfixes	IISS-487 v728 final, IISS-706/538, IISS-482 SQS (960 tests)	12–15	160.0
Phase 4 – Inventory API, X-Ray & Report	IISS-507 (562 tests, 99%), IISS-741/753/824, X-Ray specs, report writing	16–24	480.0
Total	All deliverables	24	897.0

Table A.1: Summary of internship hours by phase.